

XML: eXtensible Markup Languaje

Tecnologías XML

Esquemas XML (XML Schema)

Dr. Juan Manuel Cueva Lovelle
Departamento de Informática
Universidad de Oviedo
cueva@uniovi.es

Software y estándares para la Web

Esquema

- Introducción a XML Schema
- Ejemplos de XML Schema
- De DTDs a Schemas
- Definición de elementos simples
- Definición de atributos
- Definición de tipos
- Definición de elementos complejos
- Validación con XML Schema
- Limitaciones de XML Schema
- Generación automática de XML Schema
- Bibliografía
- Referencias Web
- Ejercicios resueltos
- Ejercicios propuestos

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Esquema

- **Introducción a XML Schema**
- Ejemplos de XML Schema
- De DTDs a Schemas
- Definición de elementos simples
- Definición de atributos
- Definición de tipos
- Definición de elementos complejos
- Validación con XML Schema
- Limitaciones de XML Schema
- Generación automática de XML Schema
- Bibliografía
- Referencias Web
- Ejercicios resueltos
- Ejercicios propuestos

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Introducción a XML Schema (I)

- Problemas de los DTDs
  - Difíciles de manipular (no son XML)
  - No son extensibles (una vez definido, no es posible añadir nuevos vocabularios a un DTD)
  - No soportan tipos de datos (ej. enteros, float, etc.)
  - No están soportados por muchas herramientas
- XML Schema
  - Permite definir esquemas de documentos
  - La sintaxis utilizada es XML
  - La sintaxis de los DTD no es XML
  - Soporta la especificación de tipos de datos y tipos definidos por el usuario
  - Soporta comprobación de restricciones numéricas

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Introducción a XML Schema (II)

- XML Schema
  - Definición de la estructura de un conjunto de documentos XML
- Validar
  - Comprobar que un documento sigue un esquema
  - La principal ventaja es evitar de errores
  - Otras aplicaciones: edición, comprensión, enlaces de programación, etc.
  - Originalmente se utilizaron los DTDs
  - Posteriormente a los DTDs se ha desarrollado XML Schema
  - Existen otras formas de validar los documentos XML
    - RELAX-NG, Schematron, etc.

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Introducción a XML Schema (III): Características (a)

- Sintaxis XML
- Soporte para Espacios de Nombres
- Mayor expresividad
  - Restricciones numéricas
  - Integridad dependientes del contexto
- Tipos de datos
  - Gran cantidad de tipos de datos predefinidos
  - Creación de tipos de datos por el usuario
- Extensibilidad
  - Inclusión/Redefinición de esquemas
  - Herencia de tipos de datos
- Soporte a Documentación

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Introducción a XML Schema (IV): Características (b)

- XML Schema permiten:
  - Describir estructura
    - Anidación
    - Multiplicidad
    - Ordenamiento
  - Describir tipos
    - Para velocidad operatoria
    - Para mejor almacenamiento
    - Para búsquedas
    - Para ingreso de datos
    - Para detectar errores
- Se almacenan en archivos **.xsd**

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Esquema

- Introducción a XML Schema
- **Ejemplos de XML Schema**
- De DTDs a Schemas
- Definición de elementos simples
- Definición de atributos
- Definición de tipos
- Definición de elementos complejos
- Validación con XML Schema
- Limitaciones de XML Schema
- Generación automática de XML Schema
- Bibliografía
- Referencias Web
- Ejercicios resueltos
- Ejercicios propuestos

Grado en Ingeniería Informática del Software

Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejemplos de XML Schema (I): *pizzas.xsd*

pizzas.xsd

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
targetNamespace="http://www.uniovi.es"
xmlns="http://www.uniovi.es"
elementFormDefault="qualified">
<xs:element name="pizzas">
  <xs:complexType>
   <xs:sequence>
     <xs:element ref="pizza" minOccurs='1' maxOccurs='unbounded'/>
   </xs:sequence>
  </xs:complexType>
</xs:element>
<xs:element name="pizza">
  <xs:complexType>
   <xs:sequence>
     <xs:element ref="ingrediente" minOccurs='1' maxOccurs='4' />
   </xs:sequence>
   <xs:attribute name="nombre"  type="xs:ID" use='required'/>
   <xs:attribute name="precio"  type="xs:integer" use='required'/>
  </xs:complexType>
 </xs:element>

<xs:element name="ingrediente">
 <xs:complexType>
  <xs:attribute name="nombre" type="xs:string" use='required' />
 </xs:complexType>
 </xs:element>
</xs:schema>
```

Elemento raíz schema

Permite especificar rangos de inclusión

Permite especificar tipos

Asociación del archivo XML con el esquema

pizzas.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<pizzas xmlns=http://www.uniovi.es
xmlns:xsi=http://www.w3.org/2001/XMLSchema-instance
xsi:schemaLocation="http://www.uniovi.es pizzas.xsd"
. . .</pizzas>
```

Grado en Ingeniería Informática del Software

Dr. Juan Manuel Cueva

Escuela de Ingeniería Informática
Universidad de Oviedo

Software y estándares para la Web

Ejemplos de XML Schema (II): **pizzas.xsd** (a)

pizzas.xsd

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema elementFormDefault="qualified"
xmlns="http://www.uniovi.es"
targetNamespace="http://www.uniovi.es"
xmlns:xs="http://www.w3.org/2001/XMLSchema">


<xs:element name="pizzas">
<xs:complexType>
<xs:sequence>
<xs:element ref="pizza" minOccurs="0" maxOccurs="unbounded"/>
</xs:sequence>
</xs:complexType>
</xs:element>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejemplos de XML Schema (III): **pizzas.xsd** (b)

pizzas.xsd continuación

```
<xs:element name="pizza">
<xs:complexType>
<xs:sequence>
<xs:element ref="ingrediente" minOccurs="0" maxOccurs="5"/>
</xs:sequence>
<xs:attribute name="nombre" type="xs:ID" use="required"/>
<xs:attribute name="precio" type="xs:integer" use="required"/>
</xs:complexType>
</xs:element>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejemplos de XML Schema (IV): **pizzas.xsd** (c)

pizzas.xsd continuación

```
<xs:element name="ingrediente">
<xs:complexType>
<xs:simpleContent>
<xs:extension base="xs:string">
<xs:attribute name="nombre" type="xs:string" use="required"/>
</xs:extension>
</xs:simpleContent>
</xs:complexType>
</xs:element>
</xs:schema>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejemplos de XML Schema (V): **pizzas.xml** (a)

pizzas.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<pizzas xsi:schemaLocation="http://www.uniovi.es pizzas.xsd"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://www.uniovi.es">

<pizza nombre="Barbacoa" precio="8">
<ingrediente nombre="Salsa Barbacoa"/>
<ingrediente nombre="Mozzarella"/>
<ingrediente nombre="Pollo"/>
<ingrediente nombre="Bacon"/>
<ingrediente nombre="Ternera"/>
</pizza>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejemplos de XML Schema (VI): **pizzas.xml** (b)

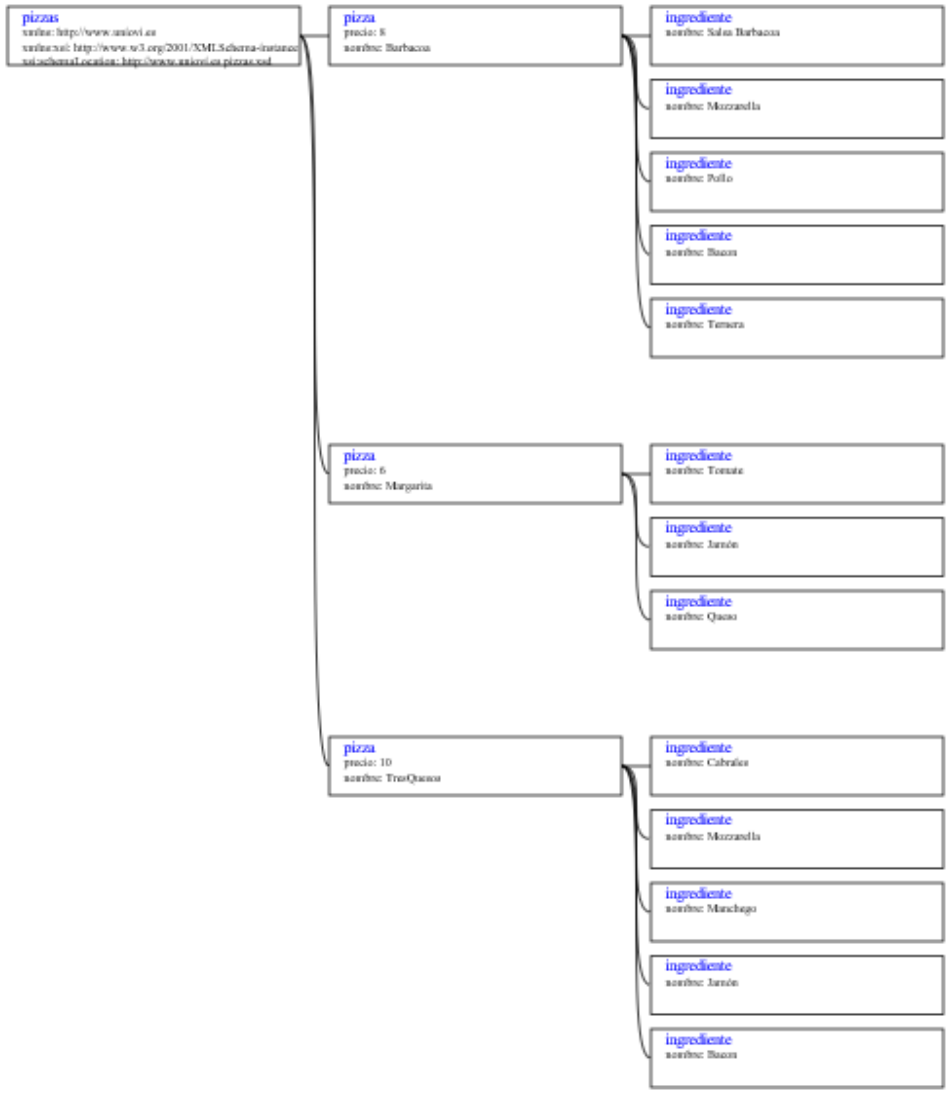
pizzas.xml **continuación**

```
<pizza nombre="Margarita" precio="6">
<ingrediente nombre="Tomate"/>
<ingrediente nombre="Jamón"/>
<ingrediente nombre="Queso"/>
</pizza>
<pizza nombre="TresQuesos" precio="10">
<ingrediente nombre="Cabrales"/>
<ingrediente nombre="Mozzarella"/>
<ingrediente nombre="Manchego"/>
<ingrediente nombre="Jamón"/><ingrediente nombre="Bacon"/>
</pizza>
</pizzas>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejemplos de XML Schema (VII): árbol **pizzas.xml**



Creado con **xml2svg.exe**

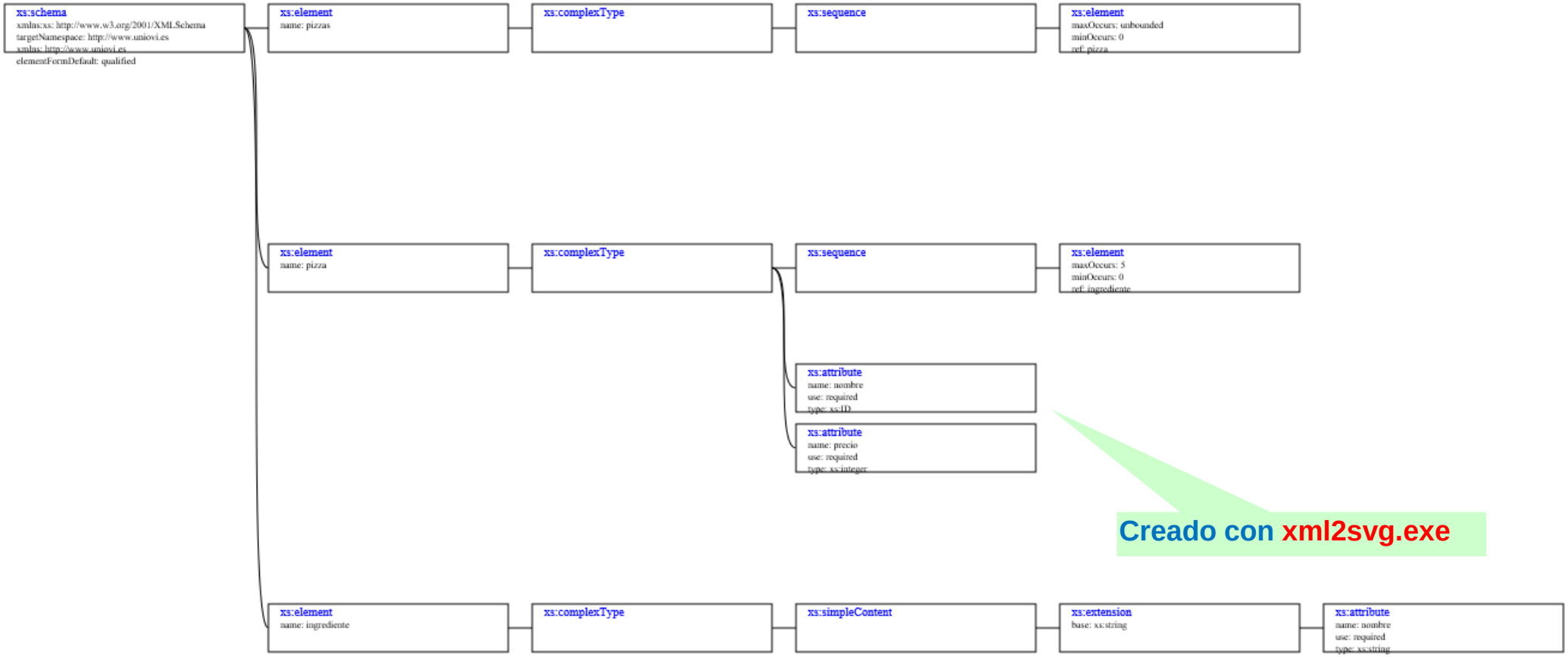
Grado en Ingeniería Informática del Software

Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejemplos de XML Schema (VIII): árbol **pizzas.xsd**



Escuela de Ingeniería Informática
Universidad de Oviedo

Software y estándares para la Web

Ejemplos de XML Schema (IX): **poema.xsd** (a)

poema.xsd

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema elementFormDefault="qualified" xmlns="http://www.uniovi.es" targetNamespace="http://www.uniovi.es"
xmlns:xs="http://www.w3.org/2001/XMLSchema">
  - <xs:element name="poesía">
      - <xs:complexType>
          - <xs:sequence>
              - <xs:element name="poema">
                  - <xs:complexType>
                      - <xs:sequence>
                          <xs:element name="titulo" type="xs:string"/>
                          <xs:element name="verso" type="xs:string" minOccurs="1" maxOccurs="unbounded"/>
                        </xs:sequence>
                        <xs:attribute name="autor" type="xs:string" use="required"/>
                        <xs:attribute name="fecha" type="xs:string" use="required"/>
                        <xs:attribute name="lugar" type="xs:string" use="optional"/>
                    </xs:complexType>
                </xs:element>
            </xs:sequence>
        </xs:complexType>
    </xs:element>
</xs:schema>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejemplos de XML Schema (X): **alba.xml**

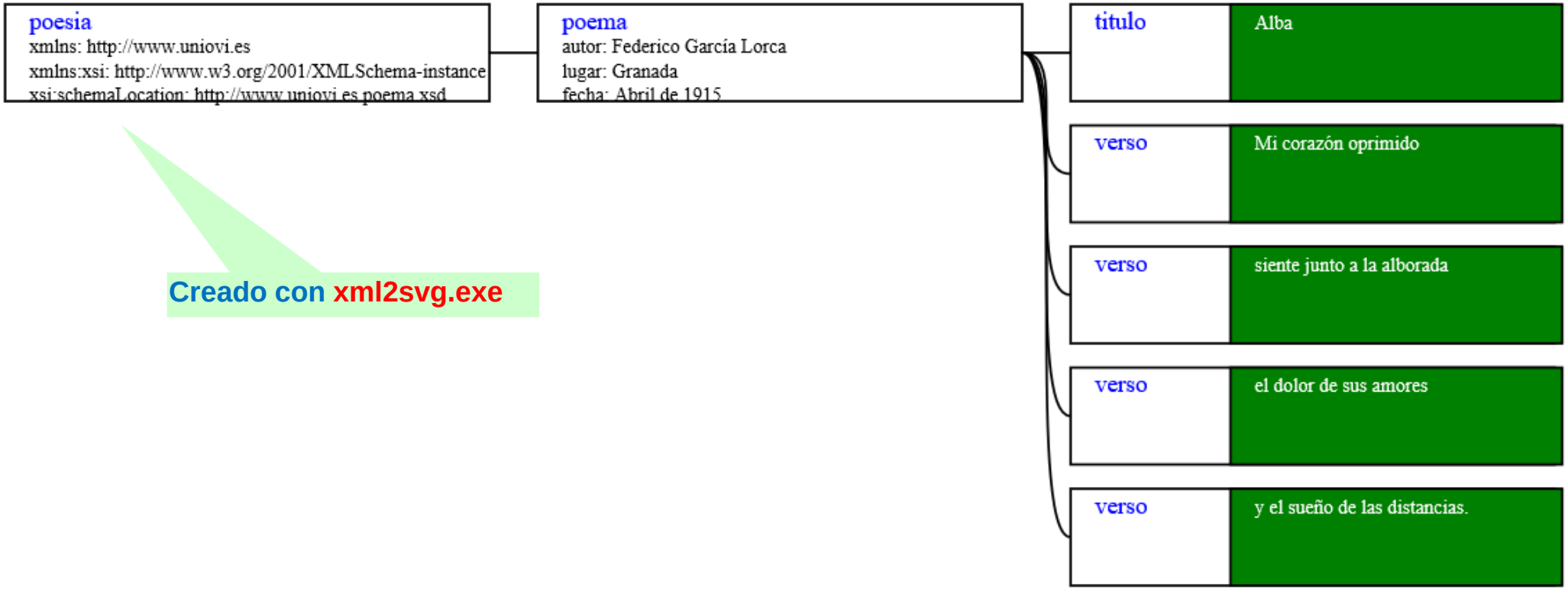
alba.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<poesía xsi:schemaLocation="http://www.uniovi.es poema.xsd"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://www.uniovi.es">
<poema fecha="Abril de 1915" lugar="Granada" autor="Federico García Lorca">
<titulo>Alba</titulo>
<verso>Mi corazón oprimido</verso>
<verso>siente junto a la alborada</verso>
<verso>el dolor de sus amores</verso>
<verso>y el sueño de las distancias.</verso>
</poema>
</poesía>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

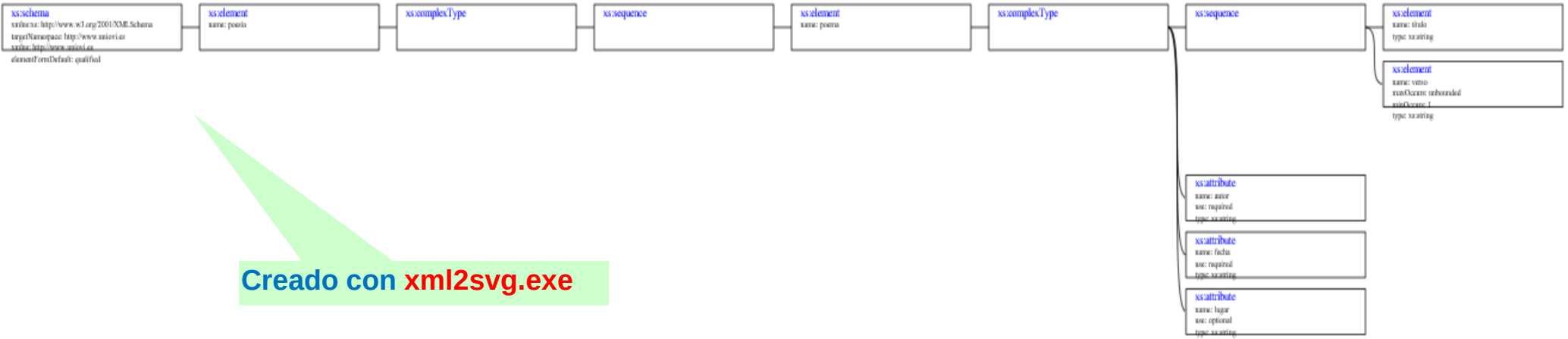
Ejemplos de XML Schema (XI): árbol **alba.xml**



Creado con **xml2svg.exe**

Software y estándares para la Web

Ejemplos de XML Schema (XII): árbol **poema.xsd**



Creado con **xml2svg.exe**

Escuela de Ingeniería Informática
Universidad de Oviedo

Software y estándares para la Web

Ejemplos de XML Schema (XIII): **libros.xsd**

libros.xsd

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xs:schema elementFormDefault="qualified" xmlns="http://www.uniovi.es" targetNamespace="http://www.uniovi.es" xmlns:xs="http://www.w3.org/2001/XMLSchema">
  - <xs:element name="libros">
      - <xs:complexType>
          - <xs:sequence>
              - <xs:element name="libro" minOccurs="1" maxOccurs="unbounded">
                  - <xs:complexType>
                      - <xs:sequence>
                          <xs:element name="titulo" type="xs:string"/>
                          <xs:element name="autor" minOccurs="1" maxOccurs="unbounded" type="xs:string"/>
                          <xs:element name="any" type="xs:gYear"/>
                        - <xs:element name="precio">
                            - <xs:complexType>
                                - <xs:simpleContent>
                                    - <xs:extension base="xs:decimal">
                                        <xs:attribute name="moneda" type="xs:string" use="required"/>
                                      </xs:extension>
                                  </xs:simpleContent>
                              </xs:complexType>
                          </xs:element>
                          <xs:element name="editorial" type="xs:string"/>
                          <xs:element name="clasificacion" type="xs:string"/>
                          <xs:element name="idioma" type="xs:string"/>
                      </xs:sequence>
                      <xs:attribute name="isbn" type="xs:ID" use="required"/>
                  </xs:complexType>
              </xs:element>
          </xs:sequence>
      </xs:complexType>
  </xs:element>
</xs:schema>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

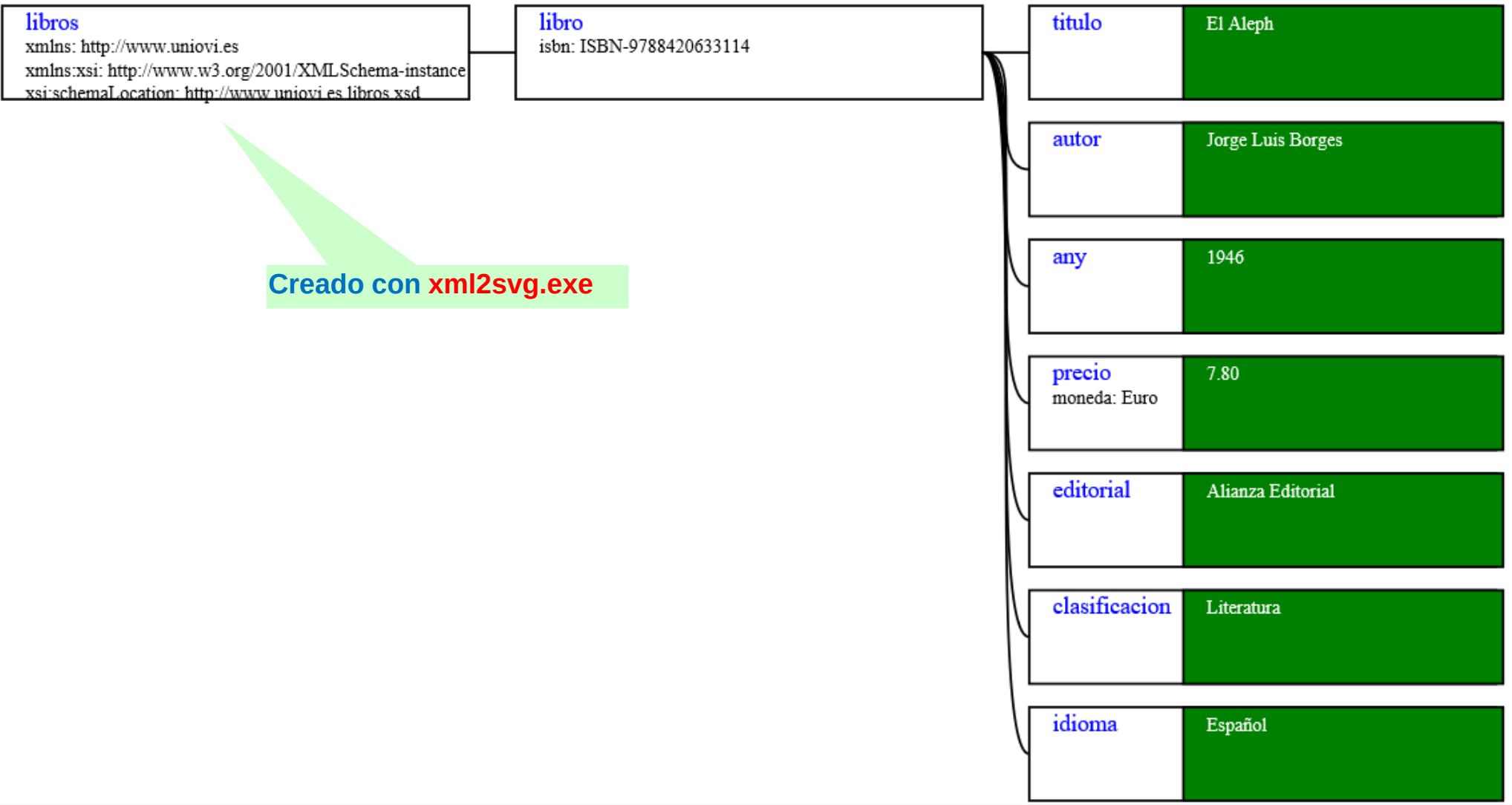
Ejemplos de XML Schema (XIV): **libros.xml**

```
<?xml version="1.0" encoding="UTF-8"?>
<libros xsi:schemaLocation="http://www.uniovi.es libros.xsd"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://www.uniovi.es">
  - <libro isbn="ISBN-9788420633114">
        <titulo>El Aleph</titulo>
        <autor>Jorge Luis Borges</autor>
        <any>1946</any>
        <precio moneda="Euro">7.80</precio>
        <editorial>Alianza Editorial</editorial>
        <clasificacion>Literatura</clasificacion>
        <idioma>Español</idioma>
    </libro>
</libros>
```

Escuela de Ingeniería Informática
Universidad de Oviedo

Software y estándares para la Web

Ejemplos de XML Schema (XV): árbol **libros.xml**



Creado con **xml2svg.exe**

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejemplos de XML Schema (XVI): árbol **libros.xsd**

Grado en Ingeniería Informática del Software



Creado con **xml2svg.exe**

Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Esquema

- Introducción a XML Schema
- Ejemplos de XML Schema
- **De DTDs a Schemas**
- Definición de elementos simples
- Definición de atributos
- Definición de tipos
- Definición de elementos complejos
- Validación con XML Schema
- Limitaciones de XML Schema
- Generación automática de XML Schema
- Bibliografía
- Referencias Web
- Ejercicios resueltos
- Ejercicios propuestos

Grado en Ingeniería Informática del Software

Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

De DTDs a Schemas (I): Ejemplo dirección.dtd y dirección.xsd

```
<!ELEMENT dirección  (empresa?, nombre, calle, ciudad, provincia, códigoPostal, teléfono+)>
<!ELEMENT           empresa              (#PCDATA)>
<!ELEMENT           nombre               (#PCDATA)>
<!ELEMENT           calle                (#PCDATA)>
<!ELEMENT           ciudad               (#PCDATA)>
<!ELEMENT           provincia            (#PCDATA)>
<!ELEMENT           códigoPostal         (#PCDATA)>
<!ELEMENT           teléfono             (#PCDATA)>
```

Grado en Ingeniería Informática del Software

dirección.xsd

```
<elementType name= "dirección" >
  <sequence>
     <elementType  name = "empresa"       minOccur="0"     maxOccur ="1"/>
     <elementType  name = "nombre"        minOccur="1"     maxOccur ="1"/>
     <elementType  name = "calle"         minOccur="1"     maxOccur ="1"/>
     <elementType  name = "ciudad"        minOccur="1"     maxOccur ="1"/>
     <elementType  name = "provincia"     minOccur="1"     maxOccur ="1"/>
     <elementType  name = "códigoPostal" minOccur="1"     maxOccur ="1"/>
     <elementType  name = "teléfono"      minOccur="1"     maxOccur ="unbounded"/>
  <sequence>
</elementType>
```

Ingeniería Informática
Universidad de Oviedo

Software y estándares para la Web

De DTDs a Schemas (II): Reglas de conversión


```
*     minOccurrs=0 maxOcurrs=unbounded
+     minOccurrs=1 maxOcurrs=unbounded
?     minOccurrs=0 maxOcurrs=1
,     xs:sequence
|     xs:choice
X     xs:element
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

De DTDs a Schemas (III): Espacio de nombres y elemento raíz

- El espacio de nombres del XML Schema es usualmente **xs**:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
```

- **Schema** es el elemento raíz del documento

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

alumnos.xsd

Ejemplo alumnos.xsd

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           targetNamespace="http://www.uniovi.es/alumnos"
           xmlns="http://www.uniovi.es/alumnos">
<xs:element name="alumnos">
 <xs:complexType>
  <xs:sequence>
   <xs:element name="alumno" minOccurs="1" maxOccurs="200"
               type="TipoAlumno"/>
  </xs:sequence>
 </xs:complexType>
</xs:element>
<xs:complexType name="TipoAlumno">
  <xs:sequence>
    <xs:element name="nombre" type="xs:string"/>
    <xs:element name="apellidos" type="xs:string"/>
    <xs:element name="nacim" type="xs:gYear"/>
  </xs:sequence>
  <xs:attribute name="dni" type="xs:string"/>
 </xs:complexType>
</xs:schema>
```

Elemento raíz schema y espacio de nombres determinado

Permite especificar rangos de inclusión

Permite especificar tipos

Grado en Ingeniería Informática del Software

Escuela de Ingeniería Informática Universidad de Oviedo

Software y estándares para la Web

Validación alumnos.xml

alumnos.xsd

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           targetNamespace="http://www.uniovi.es/alumnos"
           xmlns="http://www.uniovi.es/alumnos">
<xs:element name="alumnos">
...
</xs:schema>
```



Los espacios de nombres deben coincidir.
También puede usarse:
xsi:noNameSpaceLocation

alumnos.xml

```
<alumnos
  xmlns="http://www.uniovi.es/alumnos"
  xsi:SchemaLocation="http://www.uniovi.es/alumnos
                      alumnos.xsd"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
. . .
</alumnos>
```

Escuela de Ingeniería Informática Universidad de Oviedo

Software y estándares para la Web

Vincular un Schema a un documento XML

alumnos.xml

```
<alumnos
  xmlns="http://www.uniovi.es/alumnos"
  xsi:SchemaLocation="http://www.uniovi.es/alumnos
                      alumnos.xsd"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
. . .
</alumnos>
```

pizzas.xml

```
<pizzas xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation='pizzas.xsd'>
...
</pizzas>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Esquema

- Introducción a XML Schema
- Ejemplos de XML Schema
- De DTDs a Schemas
- **Definición de elementos simples**
- Definición de atributos
- Definición de tipos
- Definición de elementos complejos
- Validación con XML Schema
- Limitaciones de XML Schema
- Generación automática de XML Schema
- Bibliografía
- Referencias Web
- Ejercicios resueltos
- Ejercicios propuestos

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Definición de elementos simples

- Sólo puede contener texto (cualquier tipo de dato) pero no otros elementos ni atributos
- Se define como

  <xs:element name="***nombre***" type="***tipo***" **/>**
  donde
  - ***nombre*** es el nombre del elemento
  - Los valores más comunes de tipos de datos (***tipo***) son

    xs:boolean **xs:integer**
    xs:date **xs:string**
    xs:decimal **xs:time**
- Otros atributos:
  - default="***default value***" Valor por defecto de un atributo. Podría definirse otro valor.
  - fixed="***value***" Valor fijo de un atributo. Si no se define, se utiliza ése. Si se define, debe coincidir.
- *Ejemplo:* `<xs:element name="apellido" type="xs:string"/>`

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Esquema

- Introducción a XML Schema
- Ejemplos de XML Schema
- De DTDs a Schemas
- Definición de elementos simples
- **Definición de atributos**
- Definición de tipos
- Definición de elementos complejos
- Validación con XML Schema
- Limitaciones de XML Schema
- Generación automática de XML Schema
- Bibliografía
- Referencias Web
- Ejercicios resueltos
- Ejercicios propuestos

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Definición de atributos (I)

- Los atributos se declaran:

  <xs:attribute name="***nombre***" type="***tipo***" />

  Donde:
  - ***nombre*** y ***tipo*** es igual que en xs:element
- Otros atributos:
  - default="***default value***" *si no se especifica otro valor*
  - fixed="***value***“ *no se puede especificar otro valor*
  - use="optional" *el atributo no es obligatorio (opcional)*
  - use="required" *el atributo debe estar presente (obligatorio)*
- *Ejemplo:*

```
<xs:attribute name="idioma" type="xs:string"/>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Definición de atributos (II): Ejemplos de atributos


```
<xs:complexType name="Círculo">
 <xs:attribute name="radio"
               type="xs:float"
               use="required" />

 <xs:attribute name="color"
               type="Color"
               default="255 0 0"/>

 <xs:attribute name="tipo"
               type="xs:string"
               fixed="jpeg" />
</xs:complexType>
```

Por defecto los atributos son opcionales. Indicar que son obligatorios: `use="required"`

Valor por defecto de un atributo. Podría definirse otro valor.

Valor fijo de un atributo. Si no se define, se utiliza ése. Si se define, debe coincidir.

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Esquema

- Introducción a XML Schema
- Ejemplos de XML Schema
- De DTDs a Schemas
- Definición de elementos simples
- Definición de atributos
- **Definición de tipos**
- Definición de elementos complejos
- Validación con XML Schema
- Limitaciones de XML Schema
- Generación automática de XML Schema
- Bibliografía
- Referencias Web
- Ejercicios resueltos
- Ejercicios propuestos

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Definición de tipos (I)

- De los tipos básicos pueden derivarse nuevos tipos: simples o complejos.
  - Los tipos **simples** *contienen texto* y se definen mediante **xs:simpleType** .
  - y los **complejos** pueden contener cualquier combinación de contenido de elementos , información de caracteres y atributos y se definen con **xs:complexType**

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Definición de tipos (II): Tipos Simples

- No pueden contener elementos o atributos
- Pueden ser:
  - Predefinidos o *built-in* (definidos en la especificación)
    - Primitivos
    - Derivados
  - Definidos por el usuario (a partir de tipos predefinidos)

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Definición de tipos (III) Tipos Primitivos

- string
- boolean
- number, float, double
- duration, dateTime, time, date, gYearMonth, gYear, gMonthDay, gDay, gMonth
- hexBinary, base64Binary
- anyURI
- QName = Nombre cualificado con espacio de nombres
- NOTATION = Notación binaria (similar a las NOTATION de los DTD)

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

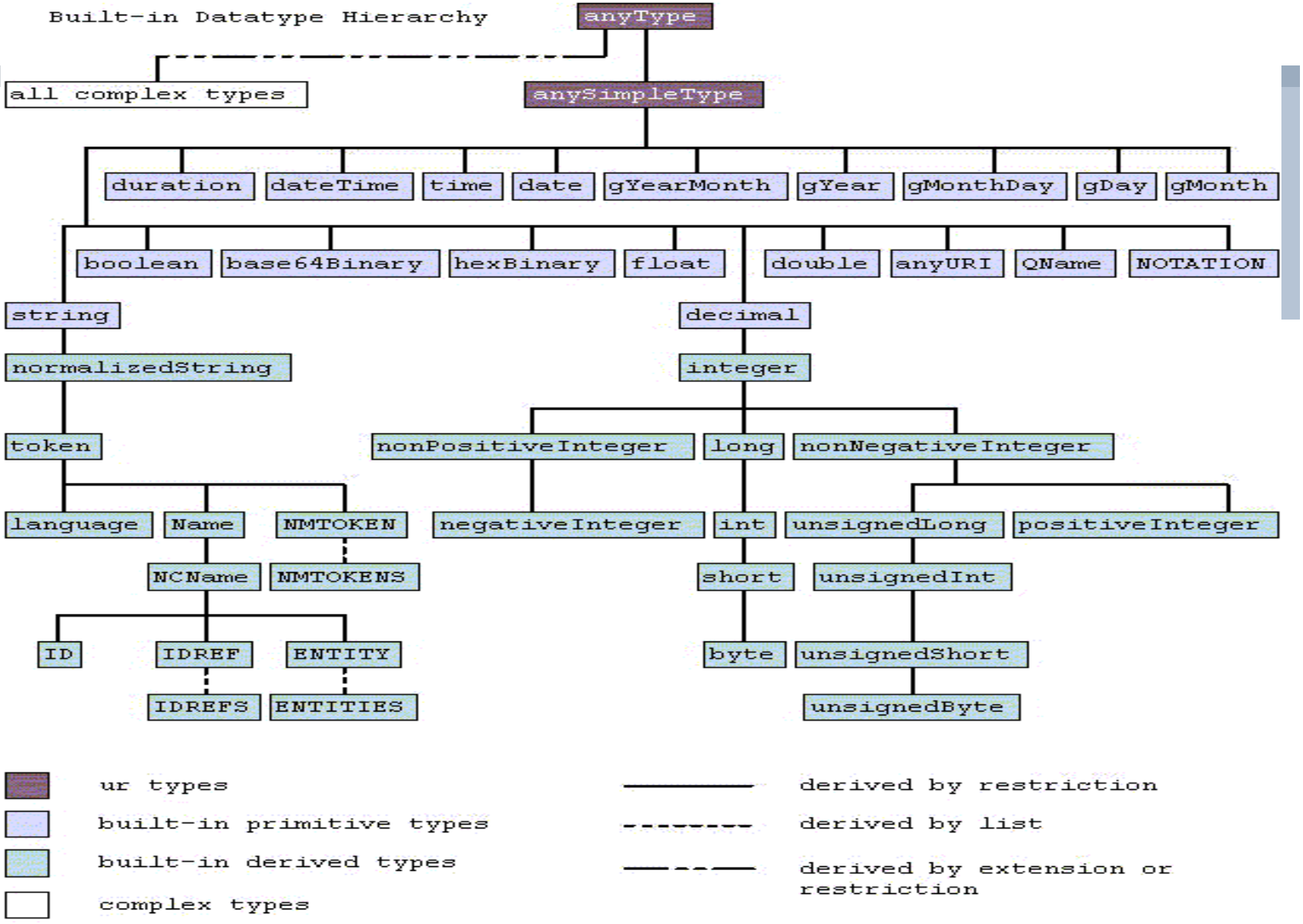
Definición de tipos (IV): Tipos Derivados

- normalizedString, token, language
- IDREFS, ENTITIES, NMTOKEN, NMTOKENS, Name, NCName, ID, IDREF, ENTITY
- integer, nonPositiveInteger, negativeInteger, long, int, short, byte, nonNegativeInteger, unsignedLong, unsignedInt, unsignedShort, unsignedByte, positiveInteger

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Definición de tipos (V): Jerarquía de tipos



Grado en Ingeniería Informática del Software

Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Definición de tipos (VI): Tipos para fechas y tiempo

- Tipo fecha (xs:date)
  - Ejemplo
    - <xs:element name="inicio" type="xs:date"/>
    - <inicio>2018-10-20</inicio>
- Zonas horarias (por ejemplo Colombia, UTC-5)
  - Ejemplo
    - <inicio>2018-10-20-05:00</inicio>
- Tipo fecha y hora (xs:dateTime)
  - Ejemplo
    - <xs:element name="inicioRuta" type="xs:dateTime"/>
    - <inicioRuta>2018-10-20T12:30:00</inicioRuta>
- Tipo duración (xs:duration)
  - La duración, tipo de datos se utiliza para especificar un intervalo de tiempo.
  - El intervalo de tiempo se especifica en la siguiente forma "**PnYnMnDTnHnMnS**" donde:
    - P indica el período (obligatorio)
    - nY indica el número de años
    - nM indica el número de meses
    - nD indica el número de días
    - T indica el inicio de una sección de tiempo (necesario si se va a especificar horas, minutos o segundos)
    - nH indica el número de horas
    - nM indica el número de minutos
    - nS indica el número de segundos
  - Ejemplo
    - <xs:element name="duracionRuta" type="xs:duration"/>
    - <duracionRuta>PT7H</duracionRuta> indica una duración de 7 horas
    - <duracionRuta>P1Y7M29DT12H</duracionRuta> indica una duración de 1 año, 7 meses, 29 días y 12 horas

Escuela de Ingeniería Informática Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Definición de tipos (VII): Facetas de Tipos

- Facetas fundamentales:
  - *equal:* Igualdad entre valores de un tipo de datos
  - *ordered:* Relaciones de orden entre valores
  - *bounded:* Límites inferiores y superiores para valores
  - *cardinality:* Define si es finito o infinito (numerable, no numerable)
  - *numeric:* Define si es numérico o no
- Facetas de restricción
  - *length, minlength, maxlength:* Longitud del tipo de datos
  - *pattern:* Restricciones sobre valores mediante expresiones regulares
  - *enumeration:* Restringe a una determinada enumeración de valores
  - *whitespace:* Define política de tratamiento de espacios (preserve/replace, collapse)
  - *(max/min)(in/ex)clusive:* Límites superiores/inferiores del tipo de datos
  - *totaldigits, fractionDigits:* número de dígitos totales y decimales

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Definición de tipos (VIII): Restricciones

- Restricciones: Permiten restringir el valor que se le puede dar a un elemento o atributo XML.
- Tipos:
  - Sobre valores
  - Sobre un conjunto de valores
  - Sobre series de valores
  - Sobre espacios en blanco

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Definición de tipos (IX): Restricciones sobre valores

- La forma general de establecer una restricción sobre un valor es:

```
<xs:element  name="name">     (o xs:attribute)
      <xs:restriction base="type">
           … Las restricciones …
       </xs:restriction>
   </xs:element>
```

- Por ejemplo:

```
<xs:element  name="edad">
  <xs:simpleType>
       <xs:restriction base="xs:integer">
            <xs:minInclusive value="0">
            <xs:maxInclusive value="140">
       </xs:restriction>
  </xs:simpleType>
</xs:element>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Definición de tipos (X): Ejemplo


```
<xs:simpleType name="mes">
<xs:restriction base="xs:integer">
  <xs:minInclusive value="1" />
  <xs:maxInclusive value="31" />
</xs:restriction>
</xs:simpleType>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Definición de tipos (XI): Enumeration

- Restringe el valor a un conjunto de valores
- Ejemplo:

```
<xs:element name=“estación">
      <xs:simpleType>
          <xs:restriction  base="xs:string">
              <xs:enumeration value=“primavera"/>
              <xs:enumeration value=“verano"/>
              <xs:enumeration value=“otoño"/>
              <xs:enumeration value=“invierno"/>
          </xs:restriction>
      </xs:simpleType>
  </xs:element>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Definición de tipos (XII): Ejemplo enumeración


```
<xs:simpleType name="Máster">
<xs:restriction base="xs:token">
 <xs:enumeration value="Ingeniería Web"/>
 <xs:enumeration value="Ciberseguridad"/>
</xs:restriction>
</xs:simpleType>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Definición de tipos (XIII): Restricciones sobre series de valores

- Para limitar el contenido del elemento XML definiendo las series de números y letras que se pueden usar usaremos patrones (expresiones regulares):

```
<xs:element name="letra">
<xs:simpleType>
  <xs:restriction base="xs:string">
    <xs:pattern value="[a-z]"/>
  </xs:restriction>
</xs:simpleType>

</xs:element>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

```
<xs:simpleType name="NIF">
 <xs:restriction base="xs:token">
   <xs:pattern value="\d{7,8}[A-Z]" />
 </xs:restriction>
</xs:simpleType>

<xs:element name="nif" type="NIF" />
```

```
<nif>9394173J</nif>
```

```
<nif>11079845M</nif>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Definición de tipos (XV): Ejemplos de expresiones regulares (a)

EXPRESIÓN REGULAR	POSIBLES VALORES
Elemento \d	Elemento 2
a*b	b, ab, aab, aaab, …
[xyz]b	xb, yb, zb
a?b	b, ab
a+b	ab, aab, aaab, …
[a-c]x	ax, bx, cx

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Definición de tipos (XVI): Ejemplos de expresiones regulares (b)

`[a-c]x`	ax, bx, cx
`[^0-9]x`	Carácter ≠ dígito seguido de x
`\Dx`	Carácter ≠ dígito seguido de x
`(pa){2}rucha`	paparucha
`.abc`	Cualquier carácter seguido de abc
`(a|b)+x`	ax, bx, aax, bbx, abx, bax,...
`a{1,3}x`	ax, aax, aaax
`\n`	Salto de línea
`\p{Lu}`	Letra mayúscula
`\p{Sc}`	Símbolo de moneda

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Definición de tipos (XVII): Restricciones sobre espacios en blanco

- whiteSpace indica lo que hay que hacer con los espacios en blanco
  - Value = "preserve“ Mantiene los espacios en blanco como están
  - Value = "replace" Cambia los espacios en blanco (saltos, tab…) por espacios
  - Value = "collapse" Reemplaza todas las secuencias de espacios en blanco por un sólo espacio en blanco

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Definición de tipos (XVIII): Restricciones en números

- minInclusive: El número debe ser ≥ ***value***
- minExclusive: El número debe ser > ***value***
- maxInclusive: El número debe ser ≤ ***value***
- maxExclusive: El número debe ser < ***value***
- totalDigits: El número debe tener exactamente ***value*** dígitos
- fractionDigits: El número no debe tener más de ***value*** dígitos después del punto decimal.

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Definición de tipos (XIX): Restricciones en strings

- length – el string debe contener exactamente ***value*** caracteres
- minLength – el string debe contener al menos ***value*** caracteres
- maxLength – el string no debe contener más de ***value*** caracteres
- pattern – el ***value*** es una expresión regular que el string debe cumplir

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Definición de tipos (XX): Listas

- Se pueden aplicar las facetas: length, maxLength, minLength, enumeration

```
<!-- Lista de componentes RGB -->
  <xs:simpleType name="ComponentesRGB">
    <xs:list itemType="ComponenteRGB"/>
  </xs:simpleType>

  <!-- Lista de exactamente 3 componentes RGB -->
  <xs:simpleType name="ColorRGB">
    <xs:restriction base="ComponentesRGB">
      <xs:length value="3"/>
    </xs:restriction>
  </xs:simpleType>
```

```
<color>255 255 0</color>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Definición de tipos (XXI): Listas. Ejemplo completo

ComponentesRGB.xsd

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- ComponentesRGB.xsd Para validar componentes RGB  -->
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           targetNamespace="http://www.uniovi.es"
           xmlns="http://www.uniovi.es"
           elementFormDefault="qualified">

  <!-- Tipo base: un componente RGB entre 0 y 255 -->
  <xs:simpleType name="ComponenteRGB">
    <xs:restriction base="xs:nonNegativeInteger">
      <xs:maxInclusive value="255"/>
    </xs:restriction>
  </xs:simpleType>

  <!-- Lista de componentes RGB -->
  <xs:simpleType name="ComponentesRGB">
    <xs:list itemType="ComponenteRGB"/>
  </xs:simpleType>

  <!-- Lista de exactamente 3 componentes RGB -->
  <xs:simpleType name="ColorRGB">
    <xs:restriction base="ComponentesRGB">
      <xs:length value="3"/>
    </xs:restriction>
  </xs:simpleType>

  <!-- Elemento raíz que contiene un color RGB -->
  <xs:element name="prueba">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="color" type="ColorRGB"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

rgb.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- rgb.xml Componentes RGB  -->
<prueba xmlns="http://www.uniovi.es"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.uniovi.es ComponentesRGB.xsd">
  <color>255 255 0</color>
</prueba>
```

Grado en Ingeniería Informática del Software

Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Definición de tipos (XXII): *xs:length*

- La faceta ***xs:length*** solamente se puede aplicar a ls siguientes tipos simples:
  - xs:anyURI, xs:base64Binary, xs:ENTITIES, xs:ENTITY,
  - xs:hexBinary, xs:ID, xs:IDREF, xs:IDREFS, xs:language,
  - xs:Name, xs:NCName, xs:NMTOKEN, xs:NMTOKENS,
  - xs:normalizedString, xs:NOTATION, xs:QName,
  - xs:string, xs:token
- También se puede aplicar a ***xs:list***, pero con una interpretación específica:
  - Para xs:list, la faceta ***xs:length*** no cuenta los caracteres, sino el ***número de ítems en la lista***.

```
<xs:simpleType name="ListaDeTresEnteros">
  <xs:list itemType="xs:integer">
    <xs:length value="3"/>
  </xs:list>
</xs:simpleType>
```

```
<miElemento>1 2 3</miElemento>
```

Escuela de Ingeniería Informática Universidad de Oviedo

Software y estándares para la Web

Definición de tipos (XXIII): Uniones


```
<xs:simpleType name="TipoNota">
<xs:union>
  <xs:simpleType>
    <xs:restriction base="xs:float">
      <xs:maxInclusive value="10" />
      <xs:minInclusive value="0" />
    </xs:restriction>
  </xs:simpleType>
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:enumeration value="No presentado" />
    </xs:restriction>
  </xs:simpleType>
</xs:union>
</xs:simpleType>

<xs:element name="nota" type="TipoNota" />
```

```
<nota> 5.75 </nota>
```

```
<nota> No presentado </nota>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Esquema

- Introducción a XML Schema
- Ejemplos de XML Schema
- De DTDs a Schemas
- Definición de elementos simples
- Definición de atributos
- Definición de tipos
- **Definición de elementos complejos**
- Validación con XML Schema
- Limitaciones de XML Schema
- Generación automática de XML Schema
- Bibliografía
- Referencias Web
- Ejercicios resueltos
- Ejercicios propuestos

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Definición de elementos complejos (I)

- Elementos que contienen otros elementos hijo o que tienen atributos.
- Se suelen dividir en 4 tipos:
  - Elementos vacíos
  - Elementos no vacíos con atributos
  - Elementos con elementos hijos
  - Elementos con elementos hijos y con “texto” o valor propio

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Definición de elementos complejos (II): Sintaxis

- Se definen como:

```
<xs:element   name="nombre">
    <xs:complexType>
        ...
    </xs:complexType>
</xs:element>
```

- Ejemplo:

```
<xsd:element name="empleado">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="nombre" type="xsd:string"/>
      <xsd:element name="apellidos" type="xsd:string"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Definición de elementos complejos (III): Indicadores

- De Orden:
  - En un determinado orden o secuencia: **sequence**
  - Pudiendo el autor del documento escoger alguno de los elementos del grupo: **choice**
  - Dar al autor total libertad respecto al orden: **all**
- De ocurrencias: cardinalidades

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Definición de elementos complejos (IV): Secuencia

<xs:sequence>

- *Secuencia: Construcción básica mediante enumeración de elementos*
  - Elementos que contienen elementos

```
<xs:complexType name="TipoAlumno">
  <xs:sequence>
    <xs:element name="nombre"    type="xs:string"/>
    <xs:element name="apellidos" type="xs:string"/>
    <xs:element name="nacim" type="xs:gYear"
                minOccurs="0" maxOccurs="1"/>
  </xs:sequence>
  <xs:attribute name="dni" type="xs:integer"/>
 </xs:complexType>
```

```
<alumno dni="10399390">
 <nombre>Manuel</nombre>
 <apellidos>Cueva Norniella</apellidos>
 <nacim>1924</nacim>
</alumno>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Definición de elementos complejos (V): Alternativa

<xs:choice>

- *Choice : Representa alternativas*
  - *¡ Atención ! : Es una o-exclusiva*

```
<xs:complexType name="Transporte">
   <xs:choice>
    <xs:element name="coche" type="xs:string"/>
     <xs:element name="tren" type="xs:string"/>
     <xs:element name="avión" type="xs:string"/>
   </xs:choice>
 </xs:complexType>
```

```
<transporte>
<coche>Renault R23</coche>
</transporte>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Definición de elementos complejos (VI): Secuencias no ordenadas

<xs:all>

- *all = Todos los elementos en cualquier orden*
- *En los DTDs se requería enumerar las combinaciones:*
  (A,B,C)|(A,C,B)|...|(C,B,A)

```
<xs:complexType name="TipoLibro">
  <xs:all>
      <xs:element name="autor" type="xs:string"/>
      <xs:element name="título" type="xs:string"/>
  </xs:all>
</xs:complexType>
<xs:element name="libro" type="TipoLibro" />
```

```
<libro>
 <autor>Juanita la Loca</autor>
 <título>No estoy loca</título>
</libro>
```

```
<libro>
  <título>El Quijote</título>
  <autor>Cervantes</autor>
 </libro>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Definición de elementos complejos (VII): Cardinalidades

- minOccurs y maxOccurs
- Se utilizan para indicar el número máximo y mínimo de veces que puede aparecer un elemento hijo de un elemento complejo
  - minOccurs= 0 : Opcionalidad
  - maxOccurs = unbounded: no existe valor límite

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Definición de elementos complejos (VIII): Ejercicio

- Transforma el modelo de contenido

$$(a \mid (b , c?))^*$$

en una definición mediante XMLSchema

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Definición de elementos complejos (IX): Solución


```
<xs:complexType>
   <xs:choice minOccurs='0' maxOccurs='unbounded'>
        <xs:element ref='a'/>
        <xs:sequence>
              <xs:element ref='b'/>
              <xs:element ref='c'
              minOccurs='0' maxOccurs='1'/>
        </xs:sequence>
   </xs:choice>
</xs:complexType>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Definición de elementos complejos (X): Contenido Mixto

- *El contenido Mixto permite mezclar texto con elementos*
- Se añade mixed="true" al elemento xs:complexType
- El texto no se menciona en el elemento y puede ir en cualquier sitio (básicamente se ignora)

```
 <xs:complexType name="TCom" mixed="true">
  <xs:choice minOccurs="0" maxOccurs="unbounded">
      <xs:element name="emph" type="xs:string"/>
  </xs:choice>
</xs:complexType>

<xs:element name="comentarios" type="TCom" />
```

```
<comentarios>
 Es un poco <emph>listillo</emph>
</comentarios>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Definición de elementos complejos (XI): Agrupaciones

- Es posible nombrar agrupaciones de elementos y de atributos para hacer referencias a ellas

```
<xs:group name="nombreApellidos">
  <xs:sequence>
    <xs:element name="nombre" type="xs:string"/>
    <xs:element name="apellidos" type="xs:string"/>
  </xs:sequence>
</xs:group>
```

```
<xs:complexType name="TipoAlumno">
 <xs:group ref="nombreApellidos" />
 <xs:element name="carrera" type="xs:string"/>
</xs:complexType>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Definición de elementos complejos (XII): Tipos "*Anónimos*" versus "*Con nombre*"

Anónimo

+ legible

```
<xs:element name="alumno">
<xs:sequence>
    <xs:element name="nombre" type="xs:string"/>
    <xs:element name="apellidos" type="xs:string"/>
  </xs:sequence>
</xs:element>
```

Con nombre

```
…
<xs:element name="alumno" type="TipoAlumno"/>
…
<xs:ComplexType name="TipoAlumno">
<xs:sequence>
    <xs:element name="nombre" type="xs:string"/>
    <xs:element name="apellidos" type="xs:string"/>
  </xs:sequence>
</xs:ComplexType>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Definición de elementos complejos (XIII): Otra posibilidad “Referencias”


```
<xs:element name="alumno">
<xs:sequence>
    <xs:element name="nombre" type="xs:string"/>
    <xs:element name="apellidos" type="xs:string"/>
  </xs:sequence>
</xs:element>
```

```
<xs:element name="alumnos">
<xs:sequence>
  <xs:element ref="alumno" />
</xs:sequence>
</xs:element>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Tipos Derivados por Extensión (I)

- Similar a las subclases de POO. Consiste en heredar elementos de un tipo base

```
<xs:complexType name="Figura" >
  <xs:attribute name="color" type="Color"/>
 </xs:complexType>

<xs:complexType name="Rectángulo">
 <xs:complexContent>
   <xs:extension base="Figura">
   <xs:attribute name="base" type="xs:float" />
   <xs:attribute name="altura" type="xs:float" />
   </xs:extension>
 </xs:complexContent>
</xs:complexType>

<xs:complexType name="Círculo">
 <xs:complexContent>
   <xs:extension base="Figura">
   <xs:attribute name="radio" type="xs:float" />
   </xs:extension>
 </xs:complexContent>
</xs:complexType>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Tipos Derivados por Extensión (II)

- Los tipos derivados pueden utilizarse en los mismos sitios que la clase base

```
<xs:element name="figuras">
   <xs:complexType>
     <xs:sequence>
      <xs:element name="figura" type="Figura"
                  maxOccurs="unbounded" />
     </xs:sequence>
   </xs:complexType>
  </xs:element>
```

```
<figuras>
<figura base="23" altura="3" xsi:type="Rectángulo" />
<figura radio="3" xsi:type="Círculo" />
</figuras>
```

Es necesario especificar el tipo mediante `xsi:type`

Tipos Abstractos

- Al igual que en la POO se pueden declarar tipos abstractos
- Mediante abstract="true" se declara un tipo como abstracto.
  - Ese tipo no puede usarse directamente
- También es posible limitar la derivación de tipos
  final="restriction"

```
<xs:complexType name="Figura" abstract="true">
  <xs:attribute name="color" type="Color"/>
 </xs:complexType>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Inclusión de Esquemas

- *<xsd:include schemaLocation="…">* permite incluir elementos de otros esquemas
  - Los elementos deben estar en el mismo espacio de nombres
  - Es como si se hubiesen tecleado todos en un mismo archivo



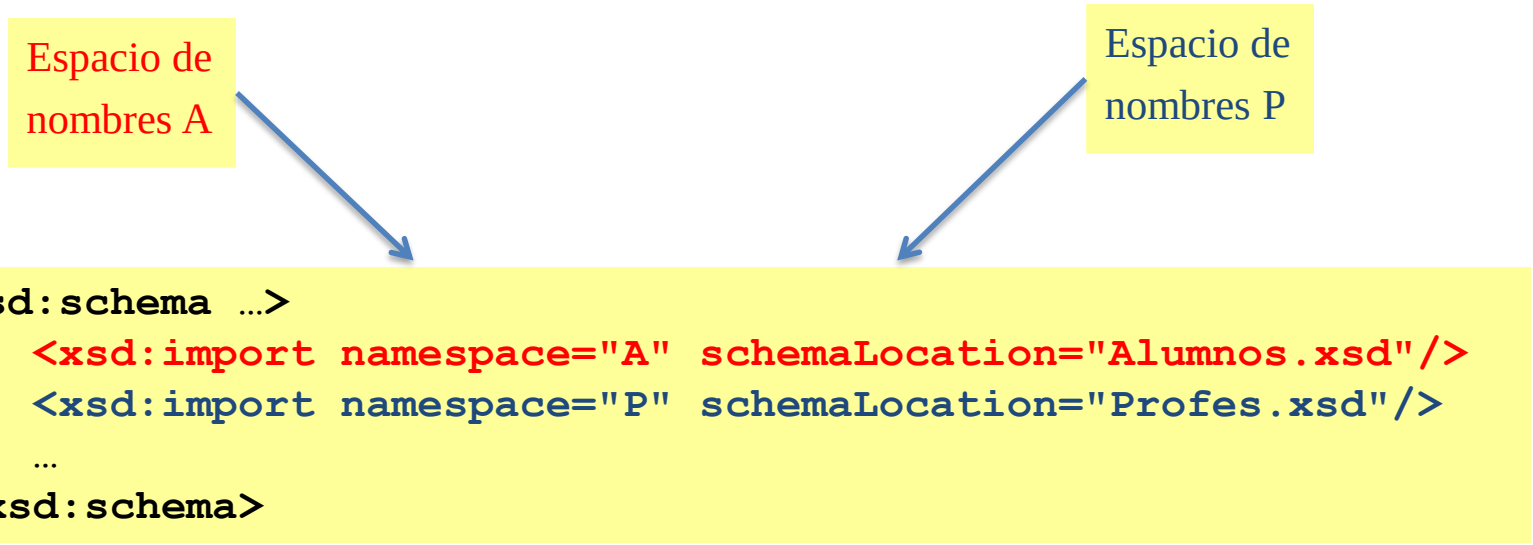

```
<xsd:schema …>
    <xsd:include schemaLocation="Alumnos.xsd"/>
    <xsd:include schemaLocation="Profesores.xsd"/>
    …
</xsd:schema>
```

Universidad.xsd

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Importación de Esquemas

- *<xsd:import namespace=…>* permite incluir elementos de otros esquemas con distintos espacios de nombres



Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Redefinición de Esquemas

- *<xs:redefine …>* es similar a *include* pero permite modificar los elementos incluidos.




```
<xs:redefine schemaLocation="Alumnos.xsd">
  <xs:complexType name="TipoAlumno">
    <xs:complexContent>
      <xs:extension base="TipoAlumno">
         <xs:sequence>
           <xs:element name="nota" type="Nota" />
         </xs:sequence>
      </xs:extension>
    </xs:complexContent>
  </xs:complexType>
</xs:redefine>
```

AlumnosConNotas.xsd

Claves y Unicidad (I)

- Los DTDs proporcionaban el atributo ID para marcar la unicidad (un valor ID era único en todo el documento)
- XML Schema tiene más posibilidades:
  - Indicar que un elemento es único (**unique**)
  - Definir atributos únicos
  - Definir combinaciones de elementos y atributos como únicos
  - Distinción entre unicidad y claves (**key**)
    - Clave = además de ser único, debe existir y no puede ser nulo.
  - Declarar el rango de un documento en el que algo es único

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Claves y Unicidad (II)


```
<xs:complexType name="Alumnos">
 <xs:sequence>
  <xs:element name="Alumno" type="TipoAlumno"/>
 </xs:sequence>
  <xs:key name="DNI">
    <xs:selector xpath="a:alumno"/>
    <xs:field xpath="a:dni"/>
  </xs:key>
</xs:complexType>
```

Es necesario incluir el espacio de nombres (XPath)

La clave puede formarse para atributos y elementos

```
<xs:key name="DNI">
    <xs:selector xpath="a:alumno"/>
    <xs:field xpath="a:nombre"/>
    <xs:field xpath="a:apellidos"/>
</xs:key>
```

Una clave puede estar formada por varios elementos

Escuela de Ingeniería Informática Universidad de Oviedo

Software y estándares para la Web

Claves y Unicidad (III)


```
<xs:complexType name="Alumnos">
 <xs:sequence>
  <xs:element name="Alumno" type="TipoAlumno"/>
 </xs:sequence>
  <xs:unique name="DNI">
    <xs:selector xpath="a:alumno"/>
    <xs:field xpath="a:dni"/>
  </xs:unique>
</xs:complexType>
```

Unique especifica que debe ser único, pero podría no existir

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Referencias a Claves

- keyref especifica que debe hacer referencia a una clave (Claves Externas)

```
<xs:element name="clase">
 <xs:sequence>
  <xs:element name="alumnos" ...
  <xs:element name="delegado" ...
 </xs:sequence>

<xs:key name="DNI">
 <xs:selector xpath="a:alumnos/a:alumno"/>
 <xs:field xpath="a:dni"/>
</xs:key>

<xs:keyref name="Delegado" refer="DNI">
  <xs:selector xpath="a:delegado"/>
  <xs:field xpath="a:dni"/>
</xs:keyref>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web
Valores Nulos

- Indicar que un elemento puede ser nulo sin estar vacío.
  - Vacío (Empty): Un elemento sin contenido
  - Nulo (Nil): Un elemento que indica que no hay valor

```
<xsd:element name="Persona">
  <xsd:complexType>
  <xsd:element name="nombre" type="xsd:NMTOKEN"/>
  <xsd:element name="primerApellido" type="xsd:NMTOKEN"/>
  <xsd:element name="segundoApellido" type="xsd:NMTOKEN"
                                      nillable="true"/>
 </xsd:complexType>
</xsd:element>
```

```
<persona>
      <nombre>John</nombre>
      <primerApellido>Smith</primerApellido>
      <segundoApell xsi:nil="true"/>
</persona>
```

El segundo apellido puede ser un **NMTOKEN** o estar indefinido

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Incluir cualquier contenido

- any indica cualquier contenido de un determinado espacio de nombres
- anyAttribute cualquier atributo de un espacio de nombres

```
<xs:complexType name="Comentario">
 <xs:sequence>
  <xs:any namespace="http://www.w3.org/1999/xhtml"
          minOccurs="1"
          processContents="skip" />
 </xs:sequence>
 <xs:anyAttribute
      namespace="http://www.w3.org/1999/xhtml"/>
</xs:complexType>
```

También puede declararse ##any, ##local, ##other

Otros valores
strict = obliga a validar
lax = valida si es posible

```
<comentarios>
  <html:p>Es un
     <html:emph>Listillo</html:emph>
  </html:p>
</comentarios>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Generalización de comentarios

- *annotation*: mecanismo para documentar los componentes de un esquema, haciendo una función similar a los comentarios
- *appinfo:* proporciona información a otras aplicaciones, lo que supone un procesamiento externo al propio documento
- *documentation:* texto o referencia a texto dentro del elemento *annotation*

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Ejemplo


```
<xs:complexType nombre="elemento">
  <xs:annotation">
      <xs:documentation>
      este elemento es una descripción de ...
      </xs:documentation>
  </xs:annotation">
</xs:complexType>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Esquema

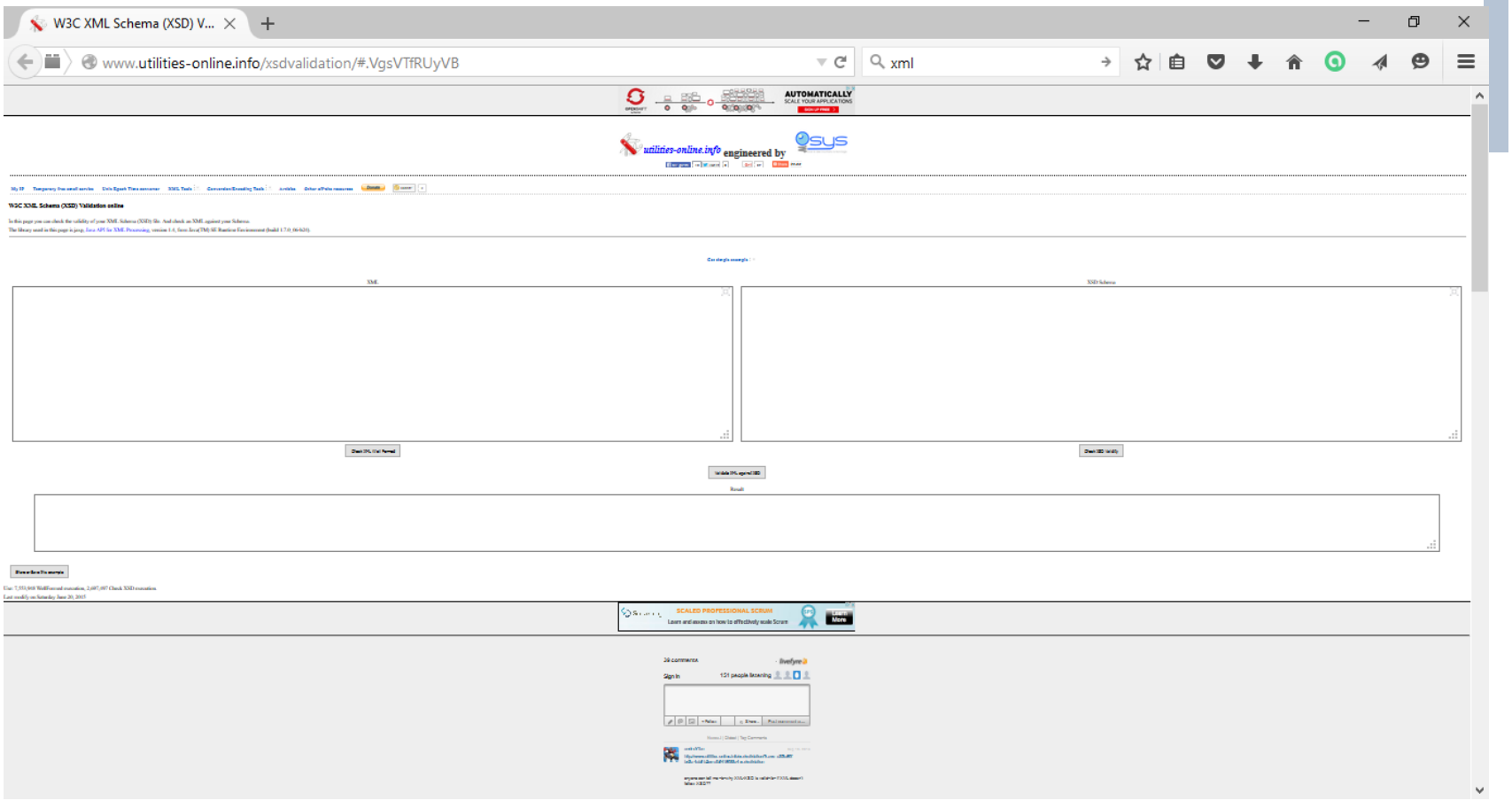
- Introducción a XML Schema
- Ejemplos de XML Schema
- De DTDs a Schemas
- Definición de elementos simples
- Definición de atributos
- Definición de tipos
- Definición de elementos complejos
- **Validación con XML Schema**
- Limitaciones de XML Schema
- Generación automática de XML Schema
- Bibliografía
- Referencias
- Ejercicios resueltos
- Ejercicios propuestos

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Validación con XML Schema(I): validador on-line

- http://www.utilities-online.info/xsdvalidation/#.VgsVTfRUyVB



Grado en Ingeniería Informática del Software

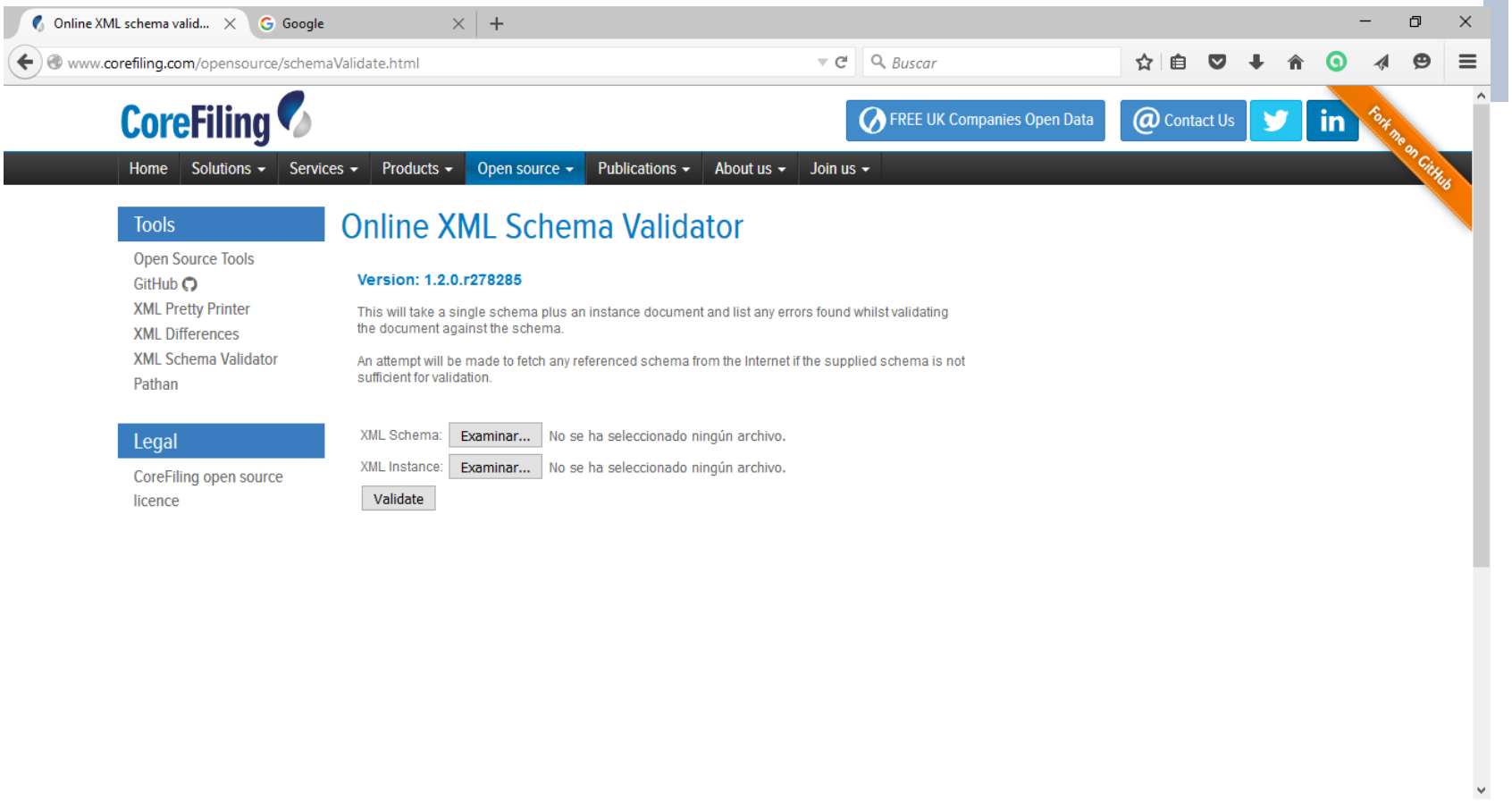
Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Validación con XML Schema(II): validador on-line

- http://www.corefiling.com/opensource/schemaValidate.html



Grado en Ingeniería Informática del Software

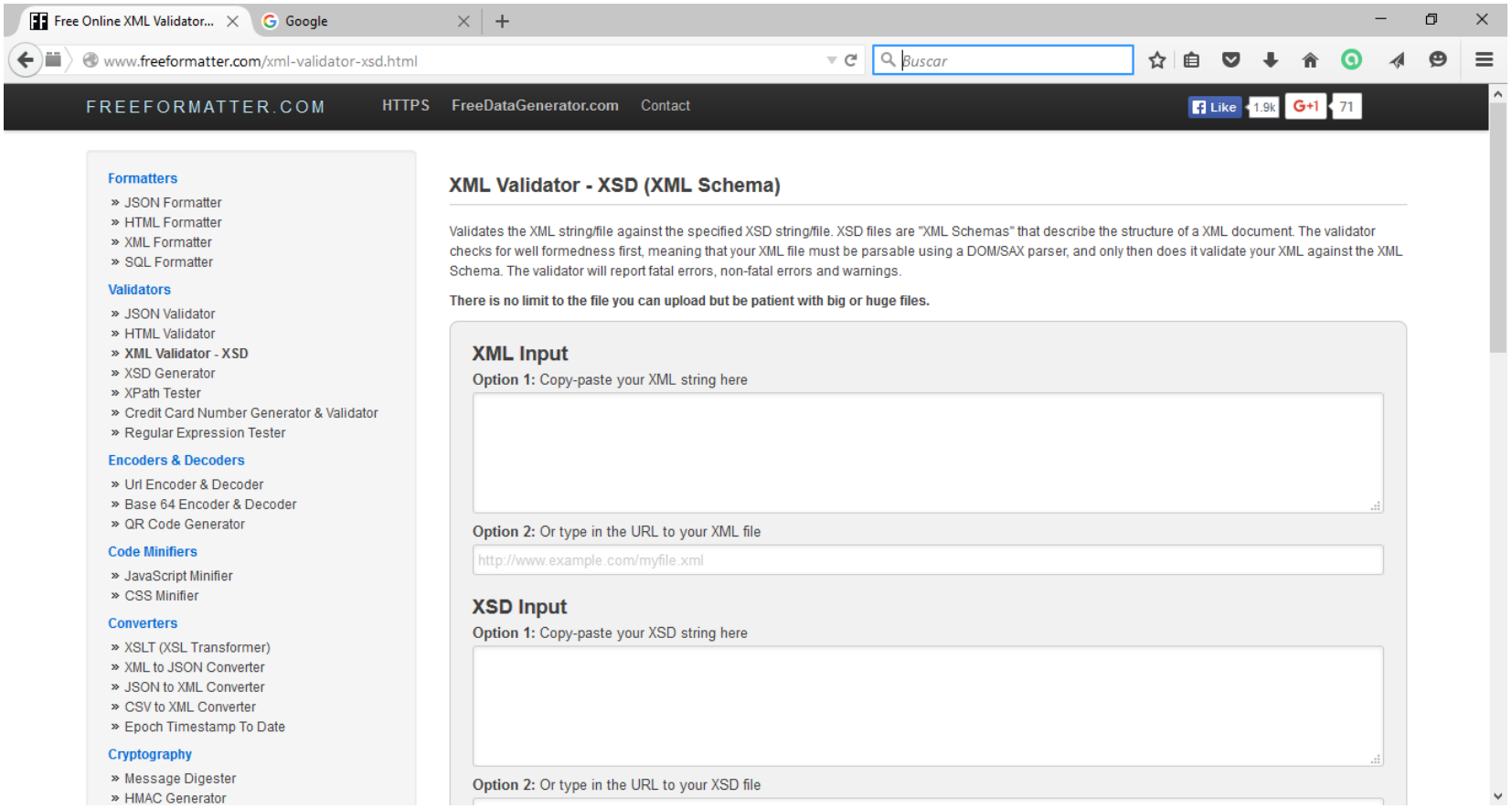
Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Validación con XML Schema(III): validador on-line

- http://www.freeformatter.com/xml-validator-xsd.html



Grado en Ingeniería Informática del Software

Escuela de Ingeniería Informática
Universidad de Oviedo

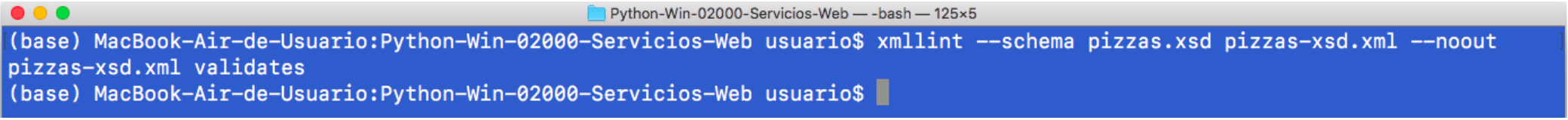
Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Validación con XML Schema(IV): en línea de comandos

- En **Linux** se puede validar un archivo .xml con un .xsd con el siguiente comando:
  - `$xmllint --schema pizzas.xsd pizzas-xsd.xml --noout`





Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

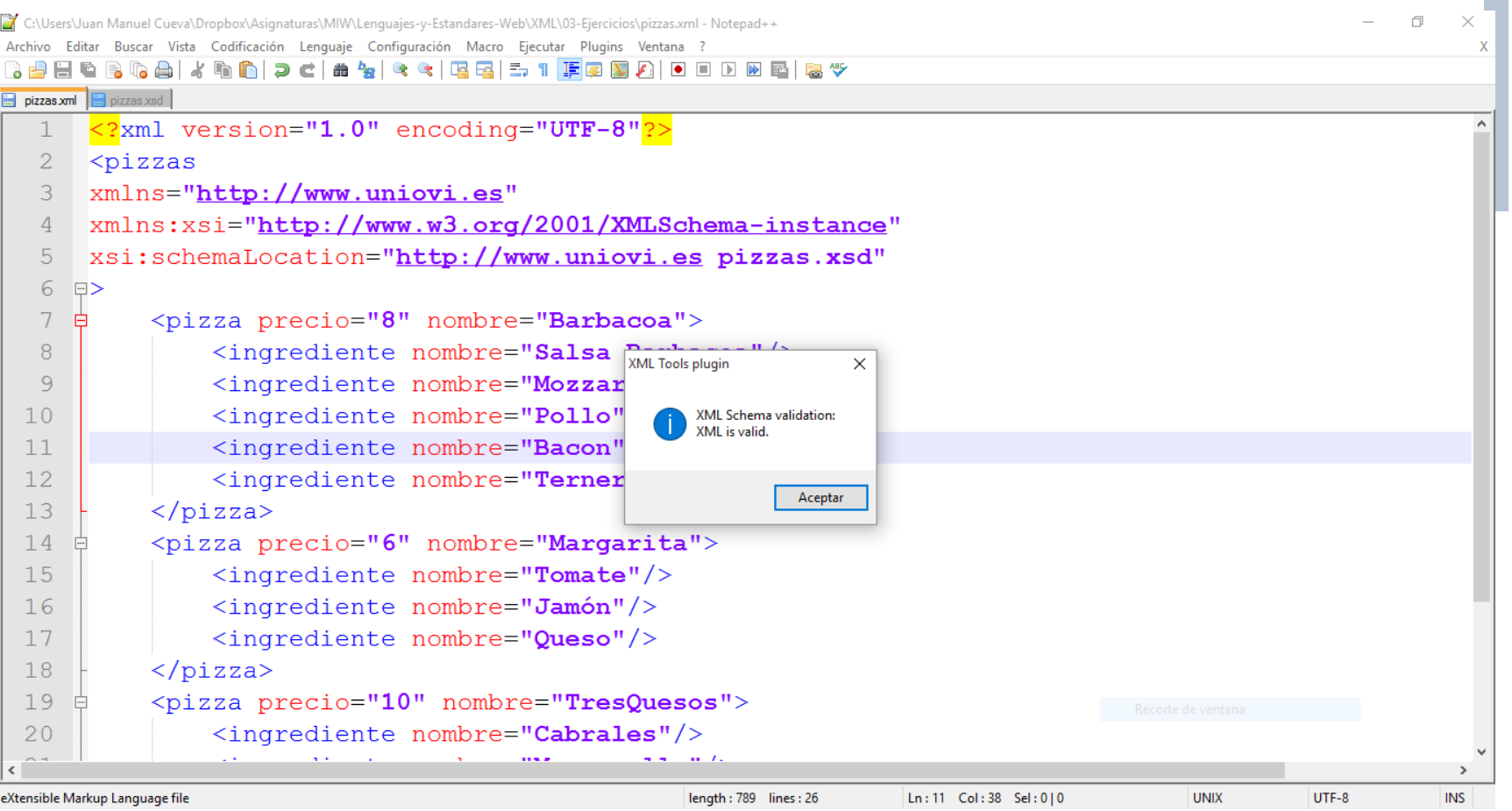
Validación con XML Schema(V): Editores con validación

- **Notepad++**
  - Editor libre para Windows
    - https://notepad-plus-plus.org/
  - Debe instalarse un plug-in para validar XML
    - XML Tools
    - Plugins >> Plugin Manager >> Show Plugin Manager >> XML Tools
- **Visual Studio Code**
    - Plug-in: **XML Language Support by Red Hat**
- **OxygenXML**
  - Editor comercial
    - http://www.oxygenxml.com/
- **XMLSpy**
  - Editor comercial
    - http://www.altova.com/xmlspy.html

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Validación con XML Schema(VI): Notepad++



Grado en Ingeniería Informática del Software

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Escuela de Ingeniería Informática
Universidad de Oviedo

Validación con XML Schema(VII): Conclusiones

- Es importante usar **múltiples** validadores
- Los validadores dan distintos mensajes de error
- Cada validador tiene una implementación diferente
- **Se debe probar con varios validadores**

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Esquema

- Introducción a XML Schema
- Ejemplos de XML Schema
- De DTDs a Schemas
- Definición de elementos simples
- Definición de atributos
- Definición de tipos
- Definición de elementos complejos
- Validación con XML Schema
- **Limitaciones de XML Schema**
- Generación automática de XML Schema
- Bibliografía
- Referencias
- Ejercicios resueltos
- Ejercicios propuestos

Grado en Ingeniería Informática del Software

Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web
Limitaciones de XML Schema

- No soporta entidades. Mecanismo para crear macros

  ```
  <!ENTITY &texto;  "Esto texto se repite muchas veces" >
  ```

  - Es necesario seguir usando los DTDs ☹
- Lenguaje de restricciones limitado
  - Ejemplo: ¿Verificar valor total = suma de valores parciales?
- Sensibilidad al contexto limitada
  - Por ejemplo: Especificar que el contenido depende del valor de un atributo

    ```
    <transporte tipo="coche"> ...</transporte>
    <transporte tipo="avión"> ...</transporte>
    ```
- Tamaño de archivos XML Schema puede ser excesivo
- Legibilidad de las especificaciones. XML Schema no siempre es muy legible
- Complejidad de la especificación:
  - Muchas situaciones/combinaciones excepcionales
- Otras propuestas: Relax-NG, Schematron, etc.

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Esquema

- Introducción a XML Schema
- Ejemplos de XML Schema
- De DTDs a Schemas
- Definición de elementos simples
- Definición de atributos
- Definición de tipos
- Definición de elementos complejos
- Validación con XML Schema
- Limitaciones de XML Schema
- **Generación automática de XML Schema**
- Bibliografía
- Referencias
- Ejercicios resueltos
- Ejercicios propuestos

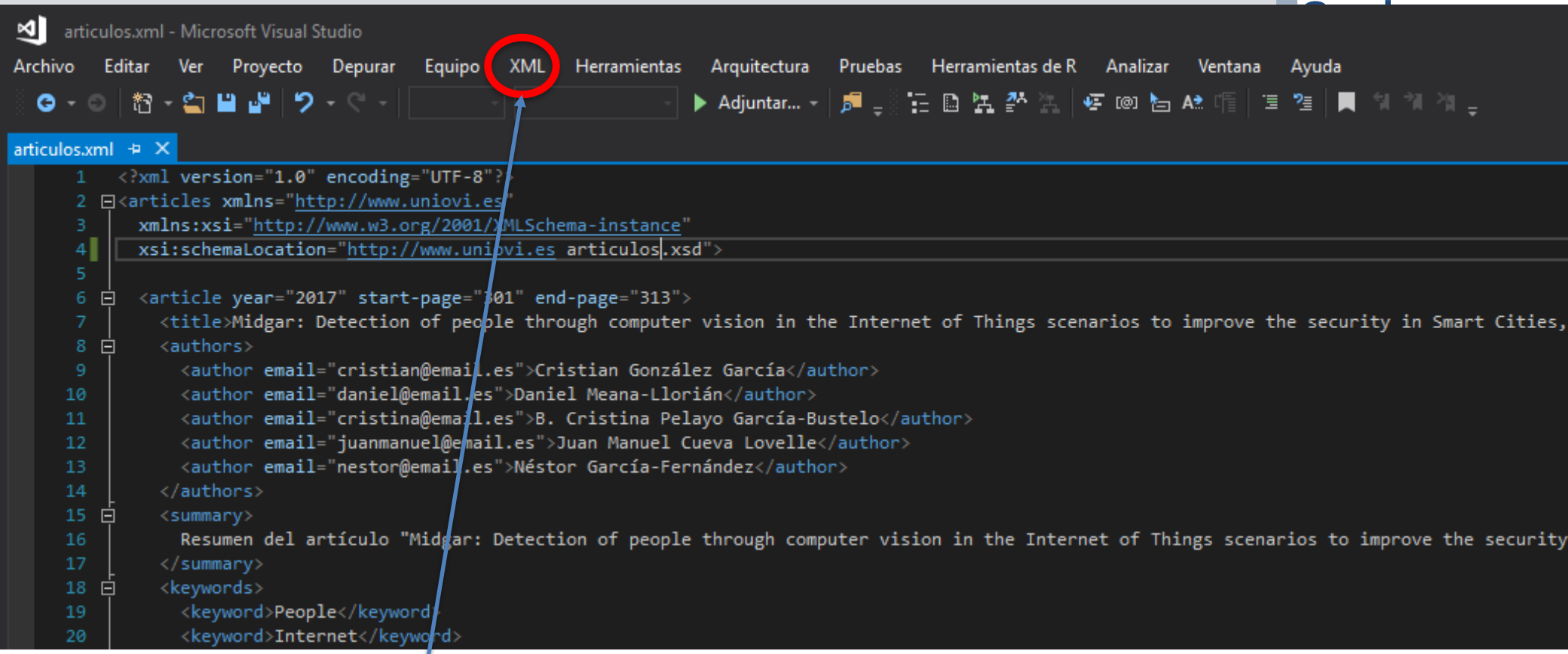
Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Generación automática de XML Schema

- Algunas herramientas generan automáticamente XML Schema a partir de un archivo XML o un archivo DTD
- El archivo generado nos ahorra tiempo de escritura
- Pero **es necesario modificarlo** debido a que es obligatorio
  - Ajustar bien los tipos de datos. Suelen aparecer demasiados *xs:string*
  - Algunos elementos o atributos opcionales pueden no aparecer si no se ha elegido lo suficientemente genérico el archivo de entrada a la herramienta
  - También es posible que no generé la codificación que deseemos

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web
Abrir archivo XML con Visual Studio Community



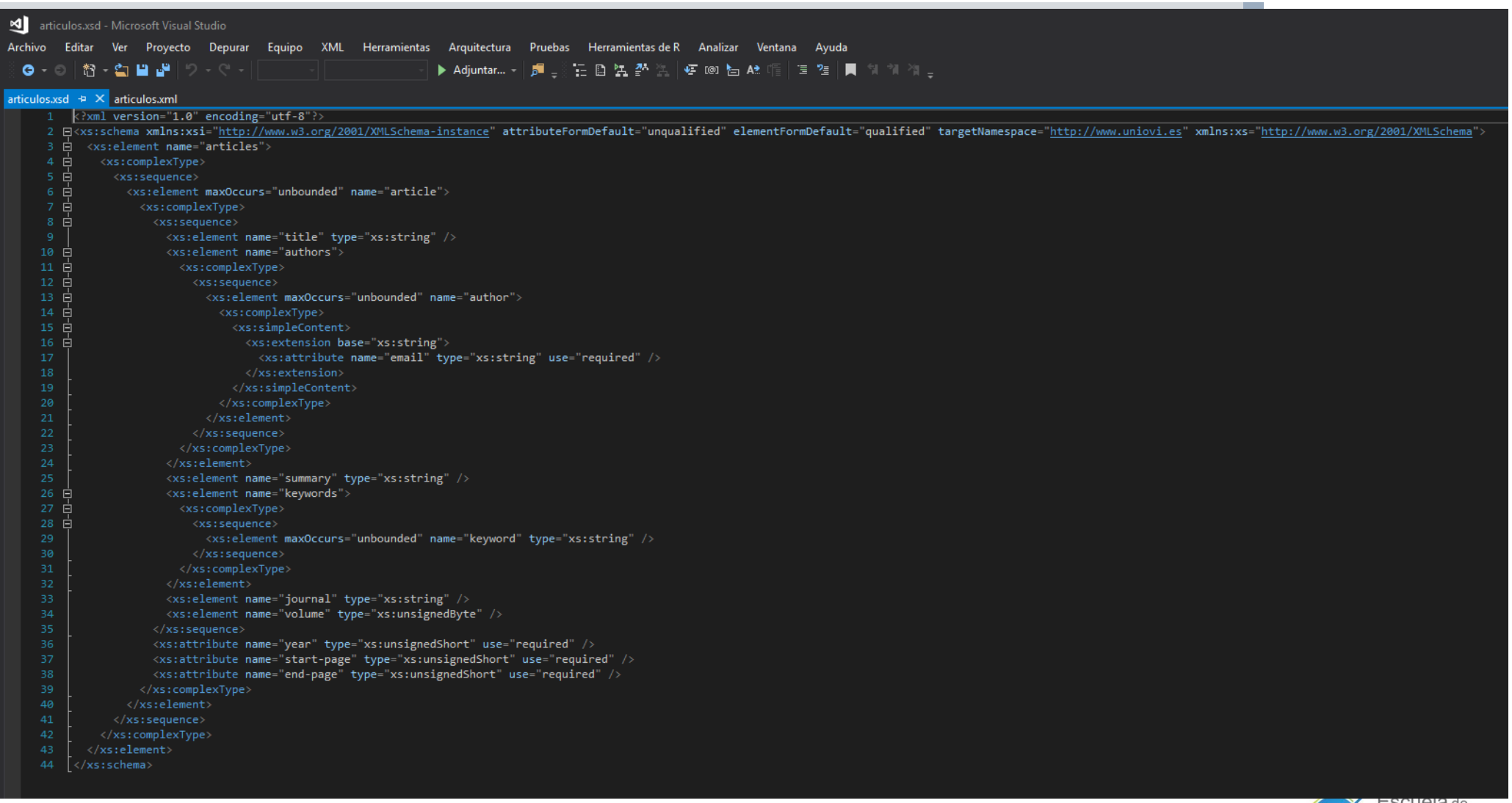
https://visualstudio.microsoft.com/es/vs/community/

Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Archivo generado con Visual Studio Community a partir de un XML

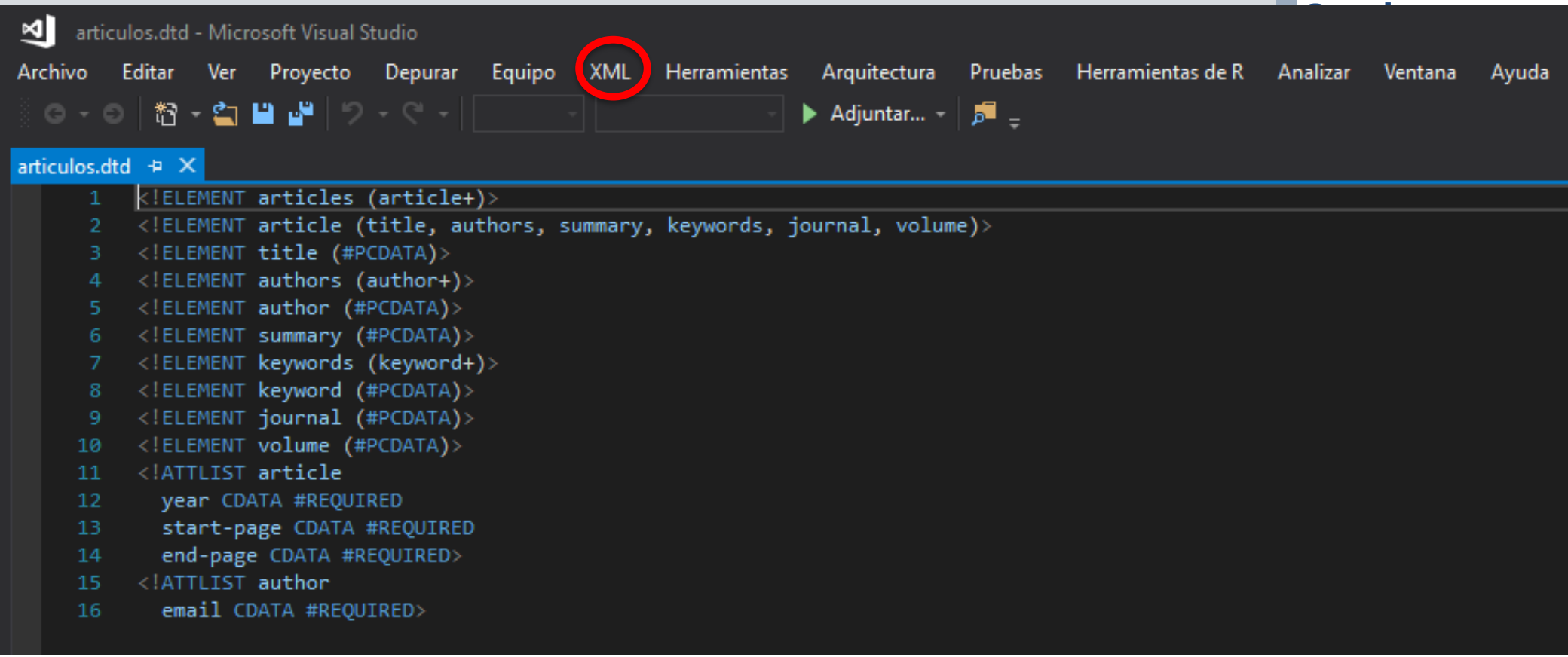


Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Abrir DTD con Visual Studio Community

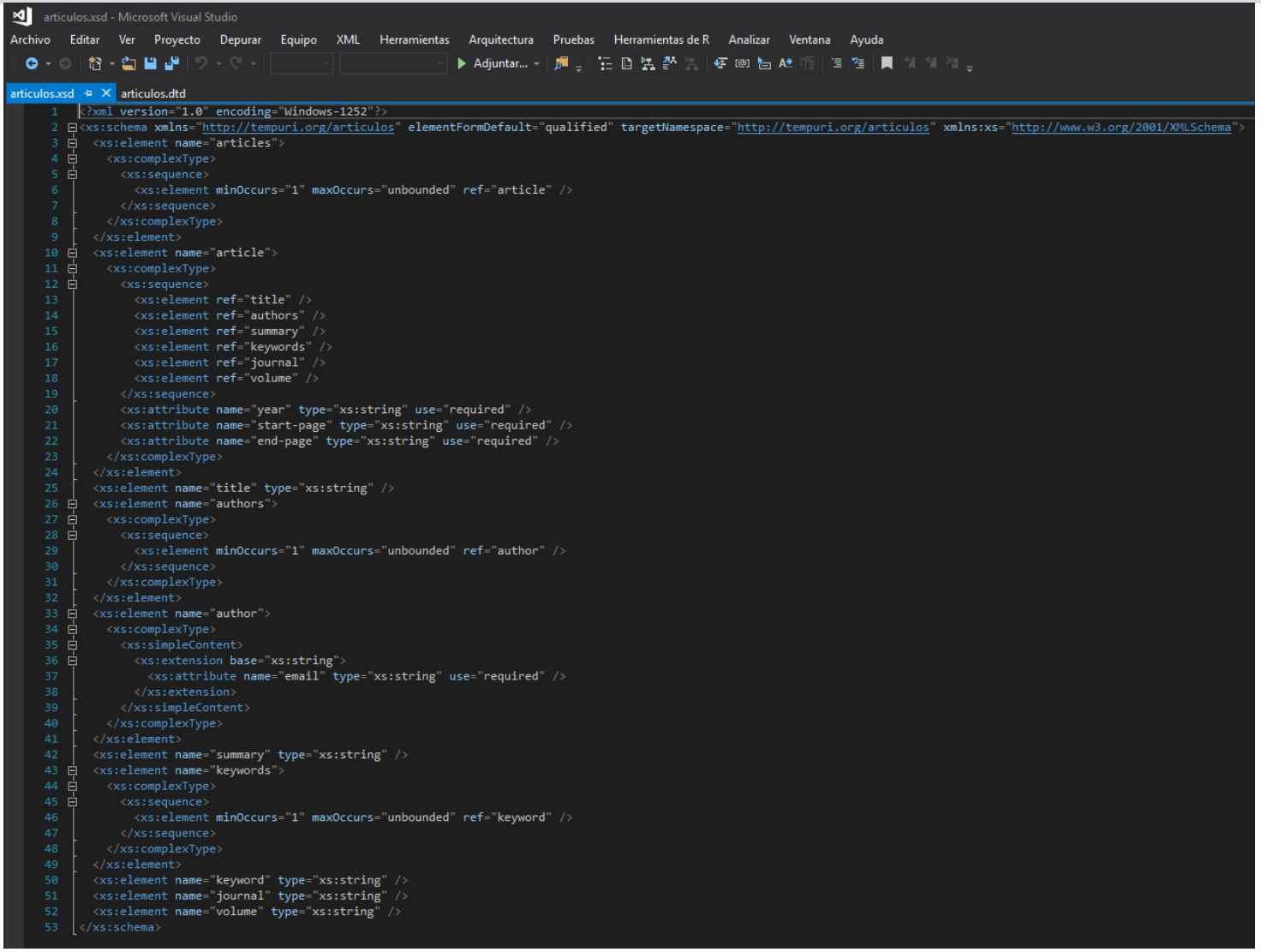


Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Archivo generado con Visual Studio Community a partir de un DTD



Grado en Ingeniería Informática del Software

Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Esquema

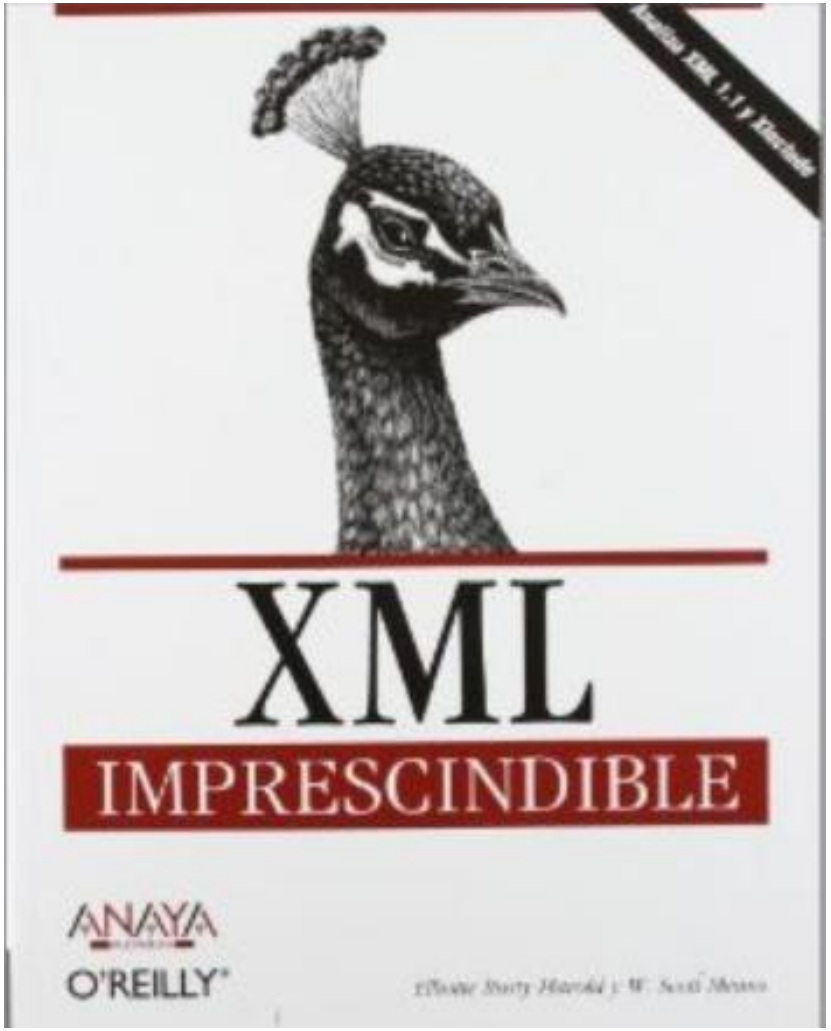
- Introducción a XML Schema
- Ejemplos de XML Schema
- De DTDs a Schemas
- Definición de elementos simples
- Definición de atributos
- Definición de tipos
- Definición de elementos complejos
- Validación con XML Schema
- Limitaciones de XML Schema
- Generación automática de XML Schema
- **Bibliografía**
- Referencias
- Ejercicios resueltos
- Ejercicios propuestos

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Bibliografía (I)

- Libro recomendado de lectura y consulta:
  - "**XML imprescindible**"
  - ANAYA/O'Reilly (2005)
  - E. Rusty Harold y W. Scott Means



Software y estándares para la Web

Bibliografía (II)

- Libro recomendado de consulta:
  - "**Beginning XML**"
  - John Wiley & Sons (2012)
  - Joe Fawcett, Liam R.E. Quin, and Danny Ayers

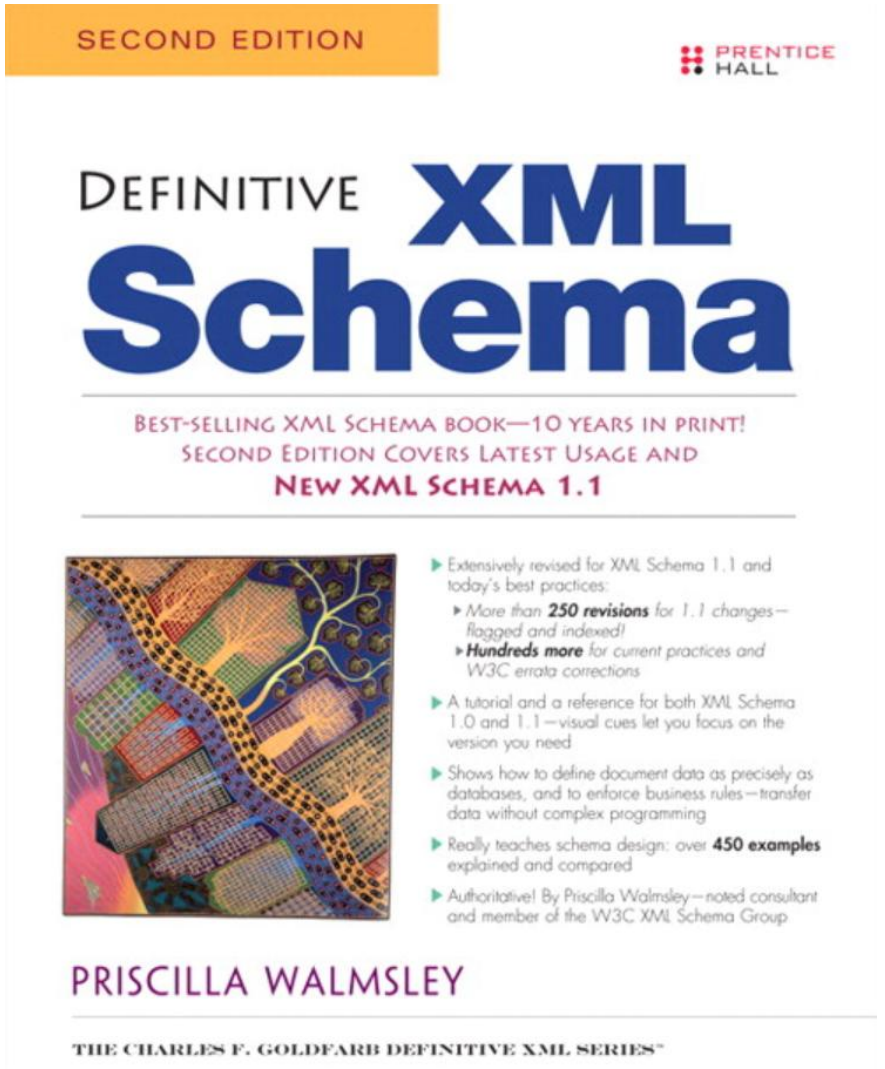


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Bibliografía recomendada

- Libro "**Definitive XML Schema**". Second Edition.
- Autora: Priscilla Walmsley
- Editorial: Prentice Hall (2012)



Escuela de Ingeniería Informática
Universidad de Oviedo

Esquema

- Introducción a XML Schema
- Ejemplos de XML Schema
- De DTDs a Schemas
- Definición de elementos simples
- Definición de atributos
- Definición de tipos
- Definición de elementos complejos
- Validación con XML Schema
- Limitaciones de XML Schema
- Generación automática de XML Schema
- Bibliografía
- **Referencias Web**
- Ejercicios resueltos
- Ejercicios propuestos

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Referencias Web (I)

- W3C recomendación XML Schema:
  - Introducción:
    - http://www.w3.org/TR/xmlschema-0/
  - Estructuras:
    - http://www.w3.org/TR/xmlschema-1/
  - Tipos de datos:
    - http://www.w3.org/TR/xmlschema-2/
  - Herramientas y recursos
    - http://www.w3.org/XML/Schema

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Referencias Web (II)

- ¿Cómo validar XML con Schema?
  - http://www.adrianmouat.com/bit-bucket/2013/11/xml-schema-validation/
- Tutoriales
  - https://www.w3schools.com/xml/schema_intro.asp
  - https://www.abrirllave.com/xsd/guion-del-tutorial.php
- Validadores de XML con XML Schema
  - http://www.utilities-online.info/xsdvalidation/#.Vgl5jTahccA
  - http://www.corefiling.com/opensource/schemaValidate.html
  - http://www.freeformatter.com/xml-validator-xsd.html

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Esquema

- Introducción a XML Schema
- Ejemplos de XML Schema
- De DTDs a Schemas
- Definición de elementos simples
- Definición de atributos
- Definición de tipos
- Definición de elementos complejos
- Validación con XML Schema
- Limitaciones de XML Schema
- Generación automática de XML Schema
- Bibliografía
- Referencias Web
- **Ejercicios resueltos**
- Ejercicios propuestos

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejercicios propuestos (I)

- Construir un **documento XML bien formado y válido usando un XML Schema** para contener artículos de revistas con los siguientes requisitos mínimos:
    - Título del artículo
    - Autores y su correo electrónico
    - Resumen
    - Palabras clave
    - Nombre de la revista
    - Número o volumen de la revista
    - Página de inicio del artículo
    - Página final del artículo
    - Año

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejercicios propuestos (I) – Solución - articulos.xsd

Grado en Ingeniería Informática del Software

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
targetNamespace="http://www.uniovi.es"
xmlns="http://www.uniovi.es"
elementFormDefault="qualified">

<xs:element name="articles">
<xs:complexType>
<xs:sequence>
<xs:element minOccurs="0" maxOccurs="unbounded" ref="article"/>
</xs:sequence>
</xs:complexType>
</xs:element>

<xs:element name="article">
<xs:complexType>
<xs:sequence>
<xs:element name="title" minOccurs="1" maxOccurs="1" type="xs:string"/>
<xs:element minOccurs="1" maxOccurs="1" ref="authors"/>
<xs:element name="summary" minOccurs="1" maxOccurs="1" type="xs:string"/>
<xs:element minOccurs="1" maxOccurs="1" ref="keywords"/>
<xs:element name="journal" minOccurs="1" maxOccurs="1" type="xs:string"/>
<xs:element name="volume" minOccurs="1" maxOccurs="1" type="xs:integer"/>
</xs:sequence>
```

Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejercicios propuestos (I) Solución - articulos.xsd - continuación

```
<xs:attribute name="year" use="required" type="xs:gYear"/>
<xs:attribute name="start-page" use="required" type="xs:integer"/>
<xs:attribute name="end-page" use="required" type="xs:integer"/>
</xs:complexType>
</xs:element>

<xs:element name="authors">
<xs:complexType>
<xs:sequence>
<xs:element minOccurs="1" maxOccurs="unbounded" ref="author"/>
</xs:sequence>
</xs:complexType>
</xs:element>

<xs:element name="author">
<xs:complexType>
<xs:simpleContent>
<xs:extension base="xs:string">
<xs:attribute name="email" type="xs:string" use="required"/>
</xs:extension>
</xs:simpleContent>
</xs:complexType>
</xs:element>
```

Grado en Ingeniería Informática del Software

Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejercicios propuestos (I) Solución - articulos.xsd - continuación

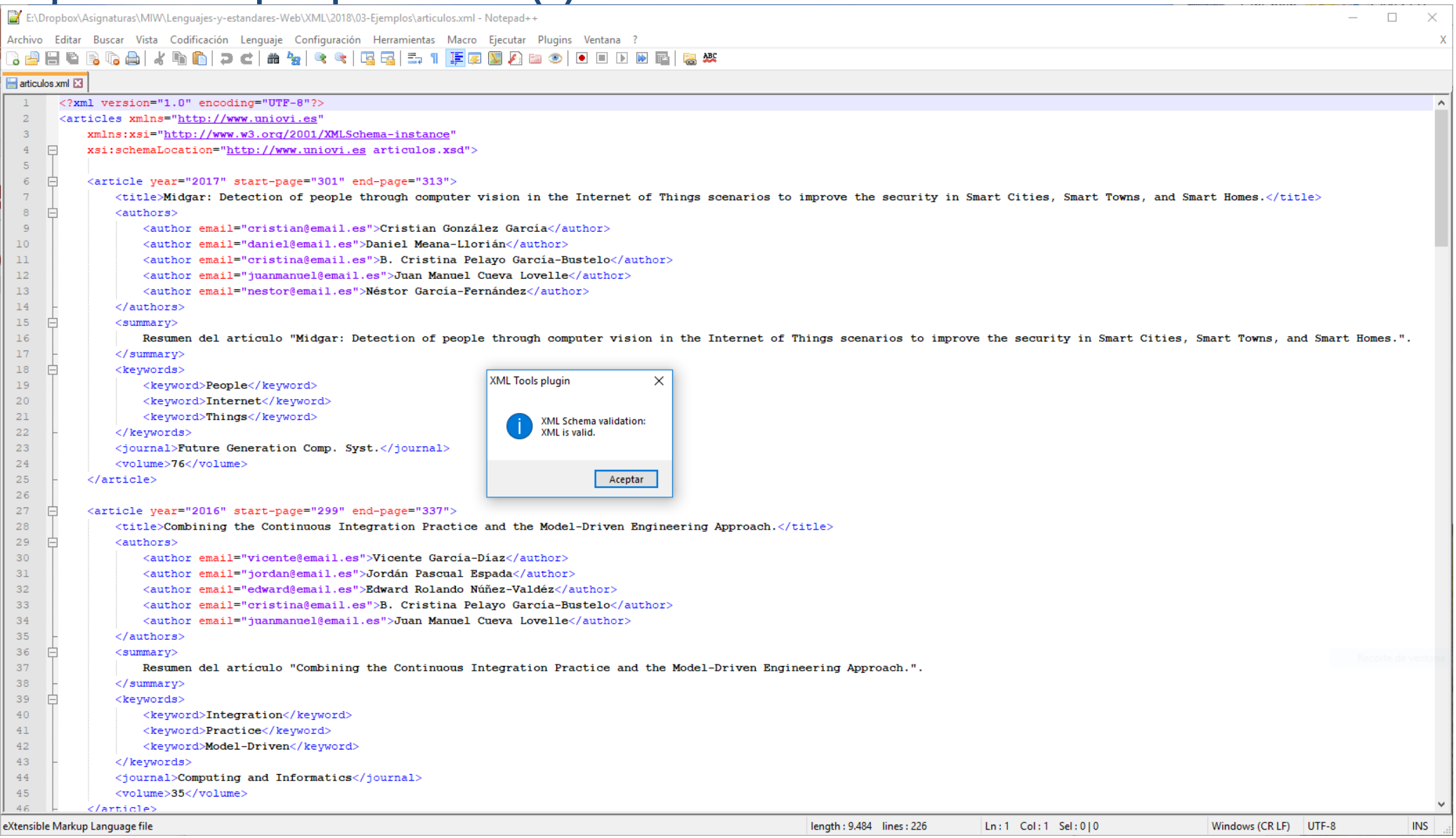

```
<xs:element name="keywords">
<xs:complexType>
<xs:sequence>
<xs:element name="keyword" minOccurs="0" maxOccurs="unbounded" type="xs:string"/>
</xs:sequence>
</xs:complexType>
</xs:element>

</xs:schema>
```


Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejercicios propuestos (I) Validación



Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejercicios propuestos (II)

- Construir un **documento XML bien formado y válido usando un XML Schema** para contener recetas de cocina con los siguientes requisitos mínimos:
  - Nombre de la receta (por ejemplo “Fabada Asturiana”)
  - Tipo de plato (postre, primer plato, entrante,. . .)
  - Ingredientes con cantidades (por ejemplo “Fabes 500 gramos”)
  - Calorías del plato (opcional)
  - Proceso de elaboración, especificado en pasos, por ejemplo:
    - Paso 1: Poner *les fabes* a remojo la noche anterior
    - Paso 2: Poner *les fabes* a cocer con agua y laurel
    - Paso 3: etc…
  - Dificultad del proceso de elaboración (por ejemplo “Fácil”, “Medio”, “Difícil”,…)
  - Tiempo de elaboración (por ejemplo “45 minutos”)
  - Elementos utilizados para la elaboración (microondas, wok, horno, freidora,. . .)
  - Origen de la receta (por ejemplo “Receta de mi abuela”, “Libro de Mª Luisa”, “Libro de las 1001 recetas”, “www.recetasMUYricas.com”)

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejercicios propuestos (II) – Solución - recetas.xsd (a)

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
targetNamespace="http://www.uniovi.es"
xmlns="http://www.uniovi.es"
elementFormDefault="qualified">

<xs:element name="cooking-recipes">
<xs:complexType>
<xs:sequence>
<xs:element minOccurs="0" maxOccurs="unbounded" ref="cooking-recipe"/>
</xs:sequence>
</xs:complexType>
</xs:element>

<xs:element name="cooking-recipe">
<xs:complexType>
<xs:sequence>
<xs:element name="name" minOccurs="1" maxOccurs="1" type="xs:string"/>
<xs:element minOccurs="1" maxOccurs="1" ref="ingredients"/>
<xs:element minOccurs="1" maxOccurs="1" ref="elaboration-process"/>
<xs:element name="origin" minOccurs="1" maxOccurs="1" type="xs:string"/>
</xs:sequence>
<xs:attribute name="type" type="dish-type" use="required"/>
<xs:attribute name="vegan" type="vegan-type" use="required"/>
</xs:complexType>
</xs:element>
```

Grado en Ingeniería Informática del Software

Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejercicios propuestos (II) – Solución - recetas.xsd (b)

```
<xs:simpleType name="dish-type">
<xs:restriction base="xs:string">
<xs:enumeration value="Entrante"/>
<xs:enumeration value="Primer plato"/>
<xs:enumeration value="Segundo plato"/>
<xs:enumeration value="Postre"/>
</xs:restriction>
</xs:simpleType>

<xs:simpleType name="vegan-type">
<xs:restriction base="xs:string">
<xs:enumeration value="Si"/>
<xs:enumeration value="No"/>
</xs:restriction>
</xs:simpleType>

<xs:element name="ingredients">
<xs:complexType>
<xs:sequence>
<xs:element minOccurs="0" maxOccurs="unbounded" ref="ingredient"/>
</xs:sequence>
</xs:complexType>
</xs:element>
```

Grado en Ingeniería Informática del Software

Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejercicios propuestos (II) – Solución - recetas.xsd (c)

```
<xs:element name="ingredient">
<xs:complexType>
<xs:simpleContent>
<xs:extension base="xs:string">
<xs:attribute name="quantity" type="quantity-type" use="required"/>
<xs:attribute name="measure" type="xs:string" use="required"/>
<xs:attribute name="calories" type="xs:integer" use="optional"/>
</xs:extension>
</xs:simpleContent>
</xs:complexType>
</xs:element>

<xs:simpleType name="quantity-type">
<xs:union memberTypes="xs:integer fraction-type" />
</xs:simpleType>

<xs:simpleType name="fraction-type">
<xs:restriction base="xs:string">
<xs:pattern value="-?\d+/-?\d+"/>
</xs:restriction>
</xs:simpleType>
```

Grado en Ingeniería Informática del Software

Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejercicios propuestos (II) – Solución - recetas.xsd (d)

```
<xs:element name="elaboration-process">
<xs:complexType>
<xs:sequence>
<xs:element minOccurs="1" maxOccurs="unbounded" ref="steps"/>
<xs:element minOccurs="0" maxOccurs="unbounded" ref="elements"/>
</xs:sequence>
<xs:attribute name="difficulty" type="difficulty-process" use="required"/>
<xs:attribute name="time-minutes" type="xs:integer" use="required"/>
</xs:complexType>
</xs:element>

<xs:simpleType name="difficulty-process">
<xs:restriction base="xs:string">
<xs:enumeration value="Fácil"/>
<xs:enumeration value="Medio"/>
<xs:enumeration value="Difícil"/>
</xs:restriction>
</xs:simpleType>

<xs:element name="steps">
<xs:complexType>
<xs:sequence>
<xs:element minOccurs="1" maxOccurs="unbounded" ref="step"/>
</xs:sequence>
</xs:complexType>
</xs:element>
```

Grado en Ingeniería Informática del Software

Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejercicios propuestos (II) – Solución - recetas.xsd (e)

```
<xs:element name="step">
<xs:complexType>
<xs:simpleContent>
<xs:extension base="xs:string">
<xs:attribute name="number" type="xs:integer" use="required"/>
</xs:extension>
</xs:simpleContent>
</xs:complexType>
</xs:element>

<xs:element name="elements">
<xs:complexType>
<xs:sequence>
<xs:element name="element" minOccurs="0" maxOccurs="unbounded" type="xs:string"/>
</xs:sequence>
</xs:complexType>
</xs:element>

</xs:schema>
```

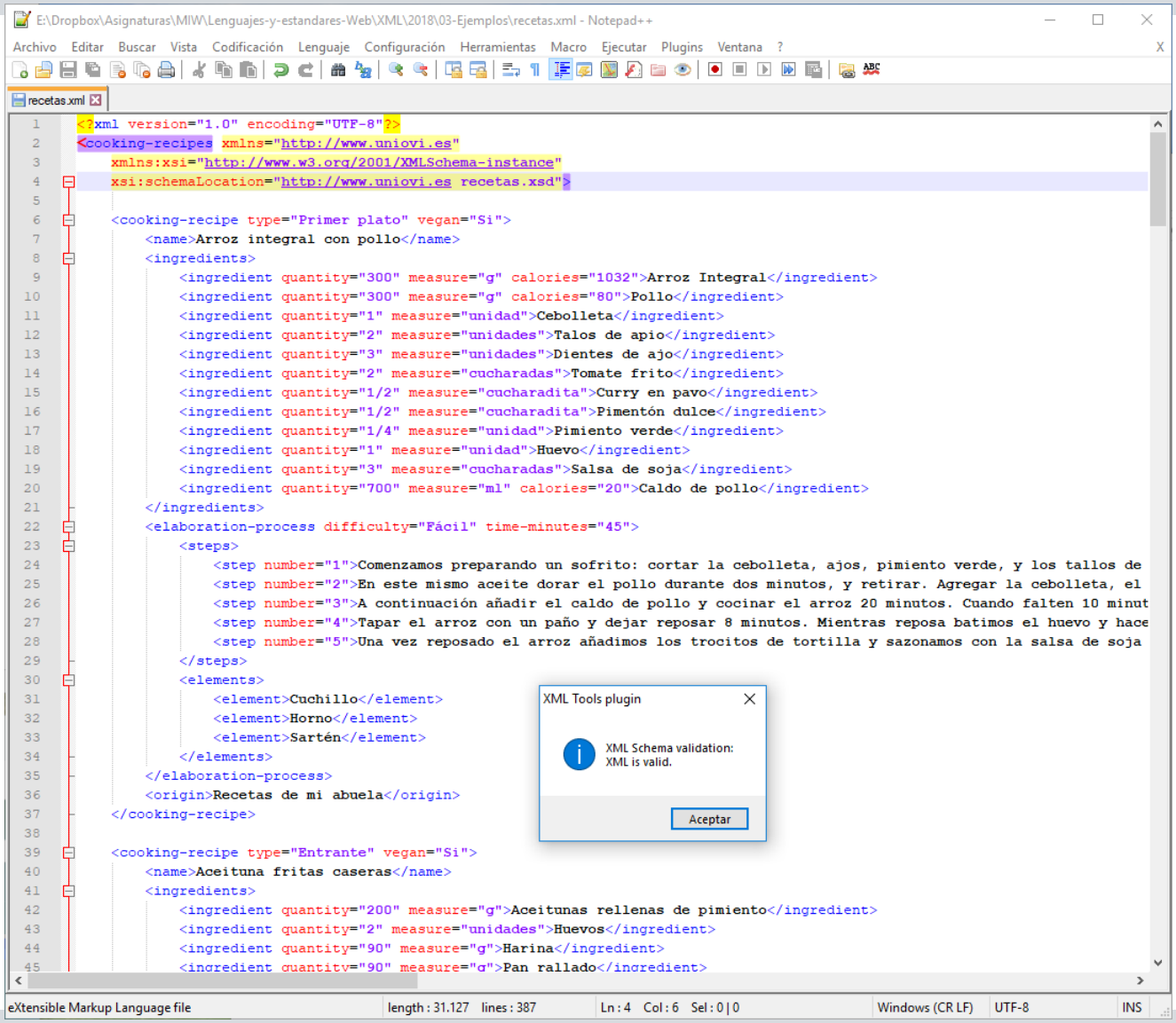
Grado en Ingeniería Informática del Software

Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejercicios propuestos (II) Validación



Grado en Ingeniería Informática del Software

Escuela de Ingeniería Informática
Universidad de Oviedo

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Esquema

- Introducción a XML Schema
- Ejemplos de XML Schema
- De DTDs a Schemas
- Definición de elementos simples
- Definición de atributos
- Definición de tipos
- Definición de elementos complejos
- Validación con XML Schema
- Limitaciones de XML Schema
- Generación automática de XML Schema
- Bibliografía
- Referencias Web
- Ejercicios resueltos
- **Ejercicios propuestos**

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejercicios propuestos (I): Monumentos prerrománicos

- Construir un **documento XML bien formado y válido usando un XML Schema** para contener monumentos del prerrománico asturiano (5 monumentos) con los siguientes requisitos mínimos:
  - Nombre del monumento (por ejemplo “Fuente de Foncalada”)
  - Tipo de monumento (por ejemplo “Arquitectura hidráulica”, “Iglesia”, “Palacio”)
  - Año de construcción aproximado (por ejemplo “1096”)
  - Constructor (por ejemplo “Desconocido”, “Ramiro I”)
  - Descripción del monumento
  - Estado del monumento (por ejemplo “bien conservado”, “mal conservado”, etc)
  - Municipio (por ejemplo “Oviedo”)
  - Dirección opcional (por ejemplo “calle Foncalada”)
  - Coordenadas geográficas: longitud, latitud y altitud
  - Galería de fotografías:
    - Fotografía 1: Por ejemplo Foncalada-01.jpg
    - Fotografía 2: Por ejemplo Foncalada-02.jpg
    - Fotografía 3: etc…

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejercicios propuestos (I): Monumentos prerrománicos - continuación

- Galería de vídeos:
  - Video 1: Por ejemplo Foncalada-01.mpeg
  - Vídeo 2: Por ejemplo Foncalada-02.mpeg
  - Video 3: etc…
- Recomendación de visita de 0 a 10 (por ejemplo “7”)
- Horario de visita
- Días de visita
- ¿Quién lo enseña?
- Referencias y bibliografía con información del monumento
  - Referencia 1: por ejemplo https://es.wikipedia.org/wiki/Foncalada
  - Referencia 2: http://prerromanicoasturiano.es/
  - Referencia 3. etc.

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejercicios propuestos (II): Rutas turísticas

- Construir un **documento XML bien formado y válido usando un XML Schema** para contener rutas turísticas (5 rutas mínimo) con los siguientes requisitos mínimos:
  - Nombre de la ruta turística (por ejemplo “Ruta por Oviedo”)
  - Tipo de ruta (por ejemplo “Arquitectura y monumentos”, “Gastronómica”, “Paisajística”, “Mixta tapas y monumentos”, “Escalada”, “Senderismo”, etc.)
  - Medio de transporte (por ejemplo “A pie”, “Automóvil”, “Bicicleta”, “Canoa”, ”Mixta a pie y tren”, etc.)
  - Fecha de inicio de la ruta (opcional)
  - Hora de inicio de la ruta (opcional)
  - Tiempo de duración de la ruta (por ejemplo “2 horas”, “3 días”, “2 semanas”,”3 meses”)
  - Agencia que gestiona la ruta (por ejemplo “Sin agencia”, “NaturAller”)
  - Descripción de la ruta
  - Personas adecuadas para la ruta (por ejemplo “Se puede ir con niños”, “Personas en buena forma física”, “tercera edad”, etc.)
  - Lugar de inicio de la ruta (por ejemplo “Oviedo”)
  - Dirección de inicio de la ruta (por ejemplo “calle Foncalada”)
  - Coordenadas geográficas de inicio de la ruta: longitud, latitud y altitud

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)

Software y estándares para la Web

Ejercicios propuestos (II): Rutas turísticas - continuación

- Referencias y bibliografía con información de la ruta (mínimo 3)
  - Referencia 1: por ejemplo https://es.wikipedia.org/wiki/Foncalada
  - Referencia 2: http://prerromanicoasturiano.es/
  - Referencia 3. etc.
- Recomendación de la ruta de 0 a 10 (por ejemplo “7”)
- Hitos de la ruta (mínimo 3 hitos):
  - Nombre del sitio
  - Descripción del sitio
  - Coordenadas geográficas del sitio: longitud, latitud, altitud
  - Distancia desde el hito anterior (las unidades se expresarán como atributos)
  - Galería de fotografías del hito (mínimo 1, máximo 5)
    - Fotografía 1: Por ejemplo Monumento.jpg
    - Fotografía 2: Por ejemplo Panorama.jpg
    - Fotografía 3: etc…
  - Galería de vídeos del hito (opcional). Mínimo 0 y máximo 3.
    - Video 1: Por ejemplo Paisaje360.mpeg o enlace a YouTube, Vimeo, etc.
    - Video 2: Por ejemplo Modelo3D.mpeg
    - Video 3: etc…

Dr. Juan Manuel Cueva Lovelle (cueva@uniovi.es)



XML: eXtensible Markup Languaje

Tecnologías XML

Esquemas XML (XML Schema)

Dr. Juan Manuel Cueva Lovelle
Departamento de Informática
Universidad de Oviedo
cueva@uniovi.es